claude-windows / 068560d3-1ca2-4387-981a-d2d6bca55414

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\068560d3-1ca2-4387-981a-d2d6bca55414\subagents\workflows\wf_ef85c152-821\agent-a3ae4e7e36332deca.jsonl`
- SHA-256: `89d8df65cb3cef34d711a5d047196e0c1a3c972620350231c50c376b7c20988f`
- Source modified: `2026-06-16T06:31:47+00:00`
- Imported at: `2026-07-05T16:48:29+00:00`
- project: `wf_ef85c152-821`
- session_id: `068560d3-1ca2-4387-981a-d2d6bca55414`

Transcript

1. user (2026-06-16T06:26:59.412Z)

LENS = DETERMINISM / cross-OS. KEY SUSPICION: the golden-crossos CI job runs `--check` + tests/test_golden_fixtures.py but NOT tests/test_golden_real_fixtures.py ? is the de-attribution/reset path therefore cross-OS-UNguarded? Also check: net_crossings integer stability across the frame*fps float round-trip; the time-origin reset edge cases (t0 from min frame, negative/zero, fps division); float rounding at 6dp vs the approx-1e-6 boundary (could a value sit exactly on a rounding cliff and flip a 6dp digit cross-OS?); dict/set iteration order in compare_expected and the JSON writers (sort_keys?); any numpy execution reachable on the CONTROL path; whether `import golden_fixtures` works on a clean clone WITHOUT numpy (experiment->classifier imports numpy at module top).

AUDIT the SHIPPED 'Golden Regression Fixtures' feature (4 merged PRs #186/#189/#191/#192) for any gap the per-PR verification missed.
Read files in the git worktree at C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-doc\ (master HEAD 8d3e185): backend/eval/golden_fixtures.py (the core); tests/test_golden_fixtures.py; tests/test_golden_real_fixtures.py; eval_baselines/fixtures/ (3 synthetic fixtures + manifest.json + README.md); .github/workflows/ci.yml (the golden-crossos job); docs/GOLDEN_REGRESSION_FIXTURES.md (§4c/§5/§6 claims). For analysis also read backend/eval/{experiment,distill_local,metrics,gt_loader,regression}.py and backend/pipeline/detectors/{tracknet_runner,rally_gate}.py.
THE FEATURE: a video-free regression harness. replay_fixture = parse_trajectory_csv -> distill_local.build_candidates(PINNED preset) -> VideoCandidates(5 shuttle cols only) -> experiment.predict(CONTROL, pure-stdlib) -> metrics.score. Synthetic Tier-0 fixtures + characterization (parse-compare: exact on int/structural, approx 1e-6 on float).
freeze_real_fixture = de-attribution (random fx_<hex> slug, metadata strip, metric-invariant time-origin reset, STRICT gt_loader, positive no-person/audio/MD5/source-path scan) + the M2 refusal gate (raise unless owned AND minors_free AND non-blank license). The golden-crossos CI job runs `--check` + tests/test_golden_fixtures.py on ubuntu/windows/macos.
ALREADY VERIFIED ? do NOT re-report as new findings: full suite 759 pass/7 skip; coverage 80.49%; ruff+mypy clean; cross-OS green on all 3 OSes for the COMMITTED fixtures; CONTROL avoids numpy *execution*.
Find what that MISSED: real gaps / bugs / edge-cases / privacy holes / determinism holes / over-claims. Cite file:line. Do NOT invent issues ? if a lens is clean, say so honestly and set verdict accordingly.

2. assistant (2026-06-16T06:27:02.397Z)

I'll start by ramping up on the repo state and then dive into the target files in the worktree.

3. user (2026-06-16T06:27:04.597Z)

8d3e185 feat(eval): golden fixtures P1 ? de-attribution/minimization tool + M2 refusal gate (#192)

4. user (2026-06-16T06:27:05.306Z)

```
1       """Golden Regression Fixtures (M1) ? video-free, CONTROL-only characterization harness.
2
3       `docs/GOLDEN_REGRESSION_FIXTURES.md` (§4, §6, §7). This module is the **single shared
4       replay code path** used by both the freeze tool and the characterization gate, and it
5       deliberately stays on the **pure-stdlib CONTROL path** (D5 in that doc): the learned
6       variants train a numpy/OpenBLAS logistic regression whose near-threshold decisions flip
7       across CPUs, so they can never anchor a cross-machine byte/parse snapshot. Here we
replay
8       only `experiment.CONTROL` (`is_learned() == False`), which keeps all candidate
intervals,
9       merges overlaps, and uses **no numpy**.
10
11      The replay reuses the already-shipped eval stack VERBATIM::
12
13          parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
14              ? VideoCandidates(name, intervals, features={5 shuttle cols},
gts=load_gt_intervals)
15              ? experiment.predict(CONTROL, vc) ? metrics.score(preds, gts, iou)
16
17      Tier-0 / privacy invariants (D3, §4c, §6 of the doc), enforced here by construction:
18
19      * Fixtures are **synthetic** ? deterministic formulae, NO randomness ? so they carry
zero
20        provenance/licensing risk and are safe to commit publicly (owner Q7 "synthetic-only").
21      * Every fixture is tagged ``kind="synthetic"``, ``minors_free=true``,
``license="synthetic"``.
22      * ``expected.json`` carries **only the 5 shuttle feature columns** ? there is, by
design, no
23        person / pose / gait / face / audio field anywhere in the schema (positive-assertion
test).
24      * The windowing preset params ``(vt, mg, mwd, max_win)`` are **pinned inside each
fixture's
25        provenance** and passed explicitly to ``build_candidates`` ? we do NOT rely on the
named
26        ``windowing.SERVED`` constant, so a future SERVED retune cannot silently churn
fixtures.
27
28      §6 honest-limits caveat (carried in the CLI / test messages, not only the docs): a green
29      suite proves only that the deterministic post-trajectory transforms are unchanged for
the
30      frozen synthetic cases ? NOT that the detector/decoder/AI handoff/DB are correct, and
NOT
31      that rally detection is good. Synthetic-tier numbers measure plumbing correctness, never
32      field quality (never cite them in QUALITY_ITERATIONS.md / the paper).
33
34      Pure + offline + stdlib. The replay imports `tracknet_runner` (which is itself stdlib-
only
35      for the parse/window math ? no TF), but pulls in no numpy/cv2 on the CONTROL path.
36      """
37      from __future__ import annotations
38
39      import argparse
40      import json
41      import math
42      import os
43      import re
44      import sys
45      from dataclasses import dataclass
46      from pathlib import Path
47      from typing import Any, Dict, List, Optional, Sequence, Tuple
48
49      # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
-------------
```

```
    from backend.eval import distill_local
    from backend.eval.experiment import CONTROL, VideoCandidates, predict
    from backend.eval.gt_loader import load_gt_intervals
    from backend.eval.metrics import score
    from backend.eval.regression import gt_hash
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

    Interval = Tuple[float, float]

    #: Bump when the fixture/expected schema changes shape. Loaders refuse a fixture whose
    #: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
    CURRENT_SCHEMA = 1

    #: Repo-root-relative fixtures dir, resolved from THIS file (never cwd) so the harness
works
    #: from any working directory / a fresh clone. ``parents[2]`` = backend/eval ? backend ?
root.
    FIXTURES_DIR: Path = Path(__file__).resolve().parents[2] / "eval_baselines" / "fixtures"

    #: The pinned SERVED windowing the synthetic fixtures freeze at: (velocity_thresh,
merge_gap,
    #: min_window_duration). Mirrors the historical SERVED values (0.02, 2.0, 2.0) but is a
LITERAL
    #: here on purpose ? the spec forbids depending on the named ``windowing.SERVED``
constant so a
    #: future retune of SERVED can never silently re-baseline a committed fixture. Each
fixture also
    #: re-states these in its provenance; the replay reads them from there.
    PINNED_VT = 0.02
    PINNED_MERGE_GAP = 2.0
    PINNED_MIN_WINDOW = 2.0
    PINNED_MAX_WINDOW = 0.0  # 0 = bounded-windowing OFF (W1), matching the SERVED default

    #: The 5 fps/resolution-invariant shuttle feature columns produced by distill_local. Re-
stated
    #: here (not imported as a mutable alias) so a fixture's expected.json is pinned to
exactly these.
    SHUTTLE_FEATURES: Tuple[str, ...] = (
        "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",
    )
    assert tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES, (
        "distill_local.FEATURE_NAMES drifted from the pinned shuttle feature set"
    )

    #: Forbidden key substrings ? biometric / identity / non-shuttle streams that MUST NOT
appear
    #: anywhere in a Tier-0 fixture (D3 / §4c). The privacy assertion scans for these.
    FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")

    _FLOAT_ROUND_DP = 6  # all written floats rounded to this many decimals (determinism)
    _FLOAT_TOL = 1e-6    # abs tolerance when parse-comparing recomputed vs committed floats

    #: A 32-hex-char MD5 (the ``video_id`` shape, ``compute_video_id``) ? the leak-scan
refuses to
    #: emit a fixture whose written text contains one (it would be an attribution back-
channel, §4c/§5).
    _MD5_RE = re.compile(r"\b[0-9a-fA-F]{32}\b")


    # ============================================================================
    # Synthetic trajectory generation (deterministic ? NO random)
    # ============================================================================


    @dataclass(frozen=True)
```

```
    class RallySpec:
        """One in-flight rally segment: the shuttle oscillates across the net midline.

        ``start_frame`` ? first frame index; ``n_frames`` ? length; ``period`` ? oscillation
        period in frames (governs net-crossing count); ``amplitude`` ? px excursion from the
        net midline along the play axis. Deterministic sine motion ? fixed
velocity/crossings.
        """

        start_frame: int
        n_frames: int
        period: float = 30.0
        amplitude: float = 400.0


    @dataclass(frozen=True)
    class GapSpec:
        """Dead-time between rallies ? what separates one window from the next.

        ``kind="still"`` ? shuttle visible but resting (velocity ? 0 ? no active frames);
        ``kind="occluded"`` ? shuttle not visible (Visibility=0 ? trajectory broken). Both
        end an active run so the next rally forms a distinct window.
        """

        start_frame: int
        n_frames: int
        kind: str = "still"  # "still" | "occluded"


    @dataclass(frozen=True)
    class FixtureSpec:
        """A full synthetic scenario: a frame-indexed sequence of rallies and gaps + its GT.

        ``gts`` are the hand-authored ground-truth rally intervals (seconds) ? the synthetic
        "labels.csv". For a synthetic fixture they line up with the rallies by construction,
        which is exactly what makes the quality-floor a *plumbing-correctness* check, not a
        field-quality one (§6).
        """

        slug: str
        description: str
        fps: float
        frame_width: float
        frame_height: float
        segments: Tuple[Any, ...]  # ordered (RallySpec | GapSpec)
        gts: Tuple[Interval, ...]
        net_midline: float = 640.0


    def _rally_points(spec: RallySpec, net_midline: float, frame_height: float
                      ) -> List[Tuple[int, int, float, float]]:
        """Deterministic in-flight shuttle: sine oscillation across the net midline (play
axis = x)."""
        rows: List[Tuple[int, int, float, float]] = []
        y_mid = frame_height / 2.0
        for i in range(spec.n_frames):
            x = net_midline + spec.amplitude * math.sin(2.0 * math.pi * i / spec.period)
            # A small orthogonal wobble keeps y-spread < x-spread so the play axis is
unambiguously x.
            y = y_mid + 30.0 * math.sin(2.0 * math.pi * i / (spec.period / 2.0))
            rows.append((spec.start_frame + i, 1, x, y))
        return rows


    def _gap_points(spec: GapSpec, net_midline: float, frame_height: float
```

```
                    ) -> List[Tuple[int, int, float, float]]:
        """Dead-time rows: resting-visible (still) or absent (occluded)."""
        rows: List[Tuple[int, int, float, float]] = []
        y_mid = frame_height / 2.0
        for i in range(spec.n_frames):
            if spec.kind == "occluded":
                rows.append((spec.start_frame + i, 0, 0.0, 0.0))
            elif spec.kind == "still":
                rows.append((spec.start_frame + i, 1, net_midline, y_mid))
            else:  # pragma: no cover - guarded by builder
                raise ValueError(f"unknown gap kind {spec.kind!r} (use 'still'|'occluded')")
        return rows


    def make_synthetic_trajectory(spec: FixtureSpec) -> List[Tuple[int, int, float, float]]:
        """Build the deterministic ``(Frame, Visibility, X, Y)`` rows for a synthetic
scenario."""
        rows: List[Tuple[int, int, float, float]] = []
        for seg in spec.segments:
            if isinstance(seg, RallySpec):
                rows.extend(_rally_points(seg, spec.net_midline, spec.frame_height))
            elif isinstance(seg, GapSpec):
                rows.extend(_gap_points(seg, spec.net_midline, spec.frame_height))
            else:  # pragma: no cover - guarded by dataclass shape
                raise TypeError(f"unknown segment type {type(seg).__name__}")
        rows.sort(key=lambda r: r[0])
        return rows


    def write_trajectory_csv(rows: Sequence[Tuple[int, int, float, float]], path: Path) ->
None:
        """Write trajectory rows as the TrackNet CSV (``Frame,Visibility,X,Y``), LF + 6dp
floats."""
        lines = ["Frame,Visibility,X,Y"]
        for frame, vis, x, y in rows:
            lines.append(f"{int(frame)},{int(vis)},{_fmt(x)},{_fmt(y)}")
        path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")


    def write_labels_csv(gts: Sequence[Interval], path: Path) -> None:
        """Write GT intervals as a ``start,end`` labels CSV (the strict gt_loader format),
LF."""
        lines = ["start,end"]
        for s, e in gts:
            lines.append(f"{_fmt(s)},{_fmt(e)}")
        path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")


    # ============================================================================
    # Built-in synthetic scenarios (the 3 committed slugs)
    # ============================================================================


    def _fr(seconds: float, fps: float) -> int:
        return int(round(seconds * fps))


    def builtin_specs() -> List[FixtureSpec]:
        """The committed synthetic scenarios. Deterministic; chosen so each exercises a
distinct
        windowing behaviour (one window / dead-time split / occlusion split)."""
        fps = 30.0
        fw, fh = 1280.0, 720.0
        net = 640.0

```

```
226         # 1) clean_single_rally: one ~4s in-flight rally ? exactly 1 candidate window,
crossings>0.
227         single = FixtureSpec(
228             slug="clean_single_rally",
229             description="One continuous ~4s rally; shuttle oscillates across the net ? 1
window.",
230             fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
231             segments=(RallySpec(start_frame=0, n_frames=120),),
232             # Window is [0.033, 3.967]; GT is the rally span the synthetic motion fills.
233             gts=((0.0, 4.0),),
234         )
235
236         # 2) multi_rally_with_deadtime: rally, 3s STILL dead-time, rally ? 2 windows split
by the
237         #    low-velocity gap (> merge_gap=2.0s) ? proves dead-time separates windows.
238         r0 = RallySpec(start_frame=0, n_frames=120)
239         gap_still = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="still")
240         r1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
241         multi = FixtureSpec(
242             slug="multi_rally_with_deadtime",
243             description="Two ~4s rallies separated by 3s of still (resting-shuttle) dead-
time ? 2 windows.",
244             fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
245             segments=(r0, gap_still, r1),
246             gts=((0.0, 4.0), (7.0, 11.0)),
247         )
248
249         # 3) occlusion_break: rally, 3s OCCLUDED (Visibility=0), rally ? 2 windows split by
the
250         #    visibility break ? proves occlusion (not just low velocity) separates windows.
251         o0 = RallySpec(start_frame=0, n_frames=120)
252         gap_occ = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="occluded")
253         o1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
254         occ = FixtureSpec(
255             slug="occlusion_break",
256             description="Two ~4s rallies separated by 3s of shuttle occlusion (Visibility=0)
? 2 windows.",
257             fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
258             segments=(o0, gap_occ, o1),
259             gts=((0.0, 4.0), (7.0, 11.0)),
260         )
261
262         return [single, multi, occ]
263
264
265     def _spec_by_slug() -> Dict[str, FixtureSpec]:
266         return {s.slug: s for s in builtin_specs()}
267
268
269     # ============================================================================
270     # The shared replay (CONTROL / pure-stdlib path)
271     # ============================================================================
272
273
274     def _fmt(x: float) -> str:
275         """Render a float with up to 6dp, no scientific notation, ``-0.0`` normalised to
``0.0``."""
276         r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
277         if r == 0.0:
278             r = 0.0
279         return f"{r:.6f}"
280
281
282     def _round(x: float) -> float:
283         r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
```

```
        return 0.0 if r == 0.0 else r


    def _pinned_proposal(prov: Optional[Dict[str, Any]]) -> Tuple[float, float, float]:
        """The windowing proposal tuple, read from the fixture provenance when present (the
pinned
        params travel with the fixture), else the module-level pinned defaults. NEVER
windowing.SERVED."""
        if prov and "windowing" in prov:
            w = prov["windowing"]
            return (float(w["vt"]), float(w["mg"]), float(w["mwd"]))
        return (PINNED_VT, PINNED_MERGE_GAP, PINNED_MIN_WINDOW)


    def replay_fixture(slug_or_dir: Any) -> Dict[str, Any]:
        """Replay one fixture through the shipped CONTROL path; return its recomputed
snapshot.

        Resolves ``slug_or_dir`` (a slug string or a fixture directory ``Path``) to its
committed
        ``trajectory.csv`` + ``labels.csv`` + ``provenance.json``, then:

            parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5
shuttle
            cols only) ? predict(CONTROL, vc) ? score(preds, gts, iou).

        Returns ``{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}],
        control_segments:[[s,e]], score:{...}, gt_hash}``. CONTROL keeps every candidate
then
        merges overlaps; NO numpy is touched.
        """
        fdir = _resolve_dir(slug_or_dir)
        slug = fdir.name
        traj_csv = fdir / "trajectory.csv"
        labels_csv = fdir / "labels.csv"
        prov = _read_json(fdir / "provenance.json") if (fdir / "provenance.json").is_file()
else None

        fps, fw = _fps_fw(prov, slug)
        iou = float(prov["iou"]) if (prov and "iou" in prov) else 0.5
        proposal = _pinned_proposal(prov)

        intervals, feature_rows = distill_local.build_candidates(
            str(traj_csv), fps=fps, frame_width=fw, proposal=proposal)

        # Build VideoCandidates DIRECTLY with ONLY the 5 shuttle feature columns. We never
call
        # fusion_compare.load_videos / ablation.load_videos ? those read c['person']
unconditionally
        # and would KeyError on these person-free fixtures (HARD CONSTRAINT 1).
        features: Dict[str, List[float]] = {
            name: [row[i] for row in feature_rows] for i, name in
enumerate(SHUTTLE_FEATURES)
        }
        gts = load_gt_intervals(str(labels_csv), strict=True)
        vc = VideoCandidates(name=slug, intervals=intervals, features=features, gts=gts)

        preds = predict(CONTROL, vc)  # CONTROL: keep all ? merge_overlaps; pure-stdlib, no
numpy
        sc = score(preds, gts, iou)

        candidates: List[Dict[str, Any]] = []
        for j, (s, e) in enumerate(intervals):
            shuttle = {name: _round(feature_rows[j][i]) for i, name in
enumerate(SHUTTLE_FEATURES)}
```

```
            # net_crossings is conceptually an integer count ? keep it exact (int), per
constraint 2.
            shuttle["net_crossings"] =
int(round(feature_rows[j][SHUTTLE_FEATURES.index("net_crossings")]))
            candidates.append({"start": _round(s), "end": _round(e), "shuttle": shuttle})

        return {
            "slug": slug,
            "schema_version": CURRENT_SCHEMA,
            "iou": _round(iou),
            "candidates": candidates,
            "control_segments": [[_round(s), _round(e)] for s, e in preds],
            "score": {
                "iou_threshold": _round(sc.iou_threshold),
                "num_pred": int(sc.num_pred),
                "num_gt": int(sc.num_gt),
                "true_positives": int(sc.true_positives),
                "false_positives": int(sc.false_positives),
                "false_negatives": int(sc.false_negatives),
                "precision": _round(sc.precision),
                "recall": _round(sc.recall),
                "f1": _round(sc.f1),
                "mean_iou": _round(sc.mean_iou),
                "mean_start_error": _round(sc.mean_start_error),
                "mean_end_error": _round(sc.mean_end_error),
            },
            "gt_hash": gt_hash(gts),
        }


    # ============================================================================
    # Freeze / refresh (expected.json + provenance.json are correct-by-construction)
    # ============================================================================


    def _provenance(spec_or_dir: Any, slug: str, fps: float, fw: float, iou: float = 0.5
                    ) -> Dict[str, Any]:
        """The committed provenance block ? pins the windowing + privacy/licensing tags
(§4c, §5)."""
        desc = ""
        if isinstance(spec_or_dir, FixtureSpec):
            desc = spec_or_dir.description
        return {
            "slug": slug,
            "schema_version": CURRENT_SCHEMA,
            "kind": "synthetic",
            "license": "synthetic",
            "minors_free": True,
            "description": desc,
            "fps": _round(fps),
            "frame_width": _round(fw),
            "iou": _round(iou),
            "windowing": {
                "vt": PINNED_VT,
                "mg": PINNED_MERGE_GAP,
                "mwd": PINNED_MIN_WINDOW,
                "max_win": PINNED_MAX_WINDOW,
            },
            "paths": {
                "trajectory": "trajectory.csv",
                "labels": "labels.csv",
                "expected": "expected.json",
            },
            "note": (
                "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only. "
```

```
                "Plumbing-correctness fixture ? NOT a measure of rally-detection quality (§6)."
            ),
        }


    def freeze_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR,
                       iou: float = 0.5) -> Dict[str, Any]:
        """Compute ``expected.json`` + ``provenance.json`` for one fixture from its COMMITTED
        ``trajectory.csv`` + ``labels.csv`` via the replay (so expected is correct-by-construction).

        Idempotent: rewrites the same bytes given the same committed inputs. Requires the trajectory
        + labels CSVs to already exist (use ``--freeze-synthetic`` to generate them first).
        """
        fdir = fixtures_dir / slug
        traj_csv, labels_csv = fdir / "trajectory.csv", fdir / "labels.csv"
        if not traj_csv.is_file() or not labels_csv.is_file():
            raise FileNotFoundError(
                f"{slug}: trajectory.csv/labels.csv missing ? run --freeze-synthetic to generate them")

        # Provenance first (it pins the windowing the replay reads), then replay ? expected.
        spec = _spec_by_slug().get(slug)
        fps, fw = (spec.fps, spec.frame_width) if spec is not None else _fps_fw_from_prov(fdir, slug)
        prov = _provenance(spec if spec is not None else fdir, slug, fps, fw, iou)
        _write_json(fdir / "provenance.json", prov)

        expected = replay_fixture(fdir)
        _write_json(fdir / "expected.json", expected)
        return expected


    def freeze_synthetic(fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
        """Generate trajectory.csv + labels.csv for every built-in scenario, freeze expected.json +
        provenance.json, and (re)write the manifest. Returns the slugs frozen."""
        specs = builtin_specs()
        fixtures_dir.mkdir(parents=True, exist_ok=True)
        slugs: List[str] = []
        for spec in specs:
            fdir = fixtures_dir / spec.slug
            fdir.mkdir(parents=True, exist_ok=True)
            rows = make_synthetic_trajectory(spec)
            write_trajectory_csv(rows, fdir / "trajectory.csv")
            write_labels_csv(spec.gts, fdir / "labels.csv")
            freeze_fixture(spec.slug, fixtures_dir=fixtures_dir)
            slugs.append(spec.slug)
        _write_manifest(specs, fixtures_dir)
        return slugs


    def refresh_snapshot(slug: Optional[str] = None, all_slugs: bool = False,
                         fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
        """Re-freeze expected.json (+ provenance.json) from the COMMITTED trajectory.csv + labels.csv ?
        NO regeneration of the trajectory/labels. ? §6: this can rubber-stamp a real regression; the
        mitigation is reviewing the visible diff + recording a reason (the CLI requires one)."""
        if all_slugs:
            targets = [d.name for d in sorted(fixtures_dir.iterdir())
```

```
                        if d.is_dir() and (d / "trajectory.csv").is_file()]
        elif slug:
            targets = [slug]
        else:
            raise ValueError("refresh_snapshot needs --slug S or --all")
        for s in targets:
            freeze_fixture(s, fixtures_dir=fixtures_dir)
        return targets


    # ============================================================================
    # P1 ? De-attribution + minimization: OWNED real trajectory+labels ? committable Tier-0
    # fixture, reusing the SAME replay_fixture snapshot. (docs/GOLDEN_REGRESSION_FIXTURES.md
    # §4c anti-attribution, §5 threat model, §7 row P1, §6 honest limits.)
    # ============================================================================


    class RefusalError(ValueError):
        """The provenance/biometric refusal gate (M2) rejected an unsafe freeze.

        A subclass of ``ValueError`` (so existing ``except ValueError`` paths still catch
it) that
        names *why* the unsafe public-fixture path was refused. The refusal is the code-
enforced
        provenance gate (§4c "provenance hard-fail gate", §7 ?): the owner-recorded /
minors-excluded
        / license flags are human-asserted, but the gate makes the unsafe path impossible to
reach
        *silently* ? a fixture cannot be written unless all three are affirmatively set.
        """


    def _new_surrogate_slug() -> str:
        """An opaque RANDOM surrogate slug ? ``fx_<16 hex>`` from ``os.urandom`` (§4c).

        Deliberately NOT the MD5 ``video_id`` and NOT ``HMAC(salt, video_id?name)`` ? a
random,
        NON-reversible id with no algebraic link to the source, so it can't be brute-forced
over a
        tiny known roster even if a salt leaked. The surrogate?source map (if kept) lives
off-git.
        """
        return f"fx_{os.urandom(8).hex()}"


    #: The fixed schema filenames the writers ALWAYS emit (as ``paths`` literals). Source-
path tokens
    #: that collide with these are NOT a leak ? exclude them so a short source stem like
``traj`` can't
    #: false-positive against the literal
``trajectory.csv``/``labels.csv``/``expected.json``.
    _SCHEMA_FILENAME_LITERALS: Tuple[str, ...] = (
        "trajectory", "trajectory.csv", "labels", "labels.csv", "expected", "expected.json",
        "provenance", "provenance.json",
    )


    def _scan_forbidden(text: str, *, where: str, source_paths: Sequence[str]) -> None:
        """POSITIVE privacy assertion (§4c, red-team ?): raise if ``text`` leaks anything
attributable.

        Scans the written JSON *text* for (a) any ``FORBIDDEN_KEYS`` biometric/identity
substring,
        (b) a 32-hex MD5 (the ``video_id`` shape), or (c) any identifying token of a source
path
```

```
        (full path / basename / stem) ? excluding the fixed schema filename literals the
writers
        always emit. The trajectory is shuttle-only by nature; this guards against a
*regression*
        re-introducing a person/audio/pose stream or an id/path back-channel ? never the
silent default.
        """
        low = text.lower()
        for key in FORBIDDEN_KEYS:
            if key in low:
                raise RefusalError(
                    f"{where}: forbidden key substring {key!r} found in written output ? "
                    f"Tier-0 fixtures are shuttle-only; person/pose/gait/face/audio/player
MUST NOT "
                    f"appear (docs/GOLDEN_REGRESSION_FIXTURES.md §4c).")
        m = _MD5_RE.search(text)
        if m:
            raise RefusalError(
                f"{where}: a 32-hex MD5 ({m.group(0)!r}) found in written output ? that is
the "
                f"video_id shape and an attribution back-channel; never write it (§4c/§5).")
        for sp in source_paths:
            if not sp:
                continue
            p = Path(sp)
            for token in (sp, p.name, p.stem):
                t = (token or "").strip().lower()
                # Skip a token that is itself a substring of a schema-literal the writers
always emit
                # (e.g. a source stem ``traj`` ? ``trajectory.csv``) ? that collision is not
a leak.
                if not t or any(t in lit for lit in _SCHEMA_FILENAME_LITERALS):
                    continue
                if t in low:
                    raise RefusalError(
                        f"{where}: source-path token {token!r} found in written output ? the
source "
                        f"filename/path must never be persisted (§4c metadata strip).")


    def _reset_trajectory_rows(points: Sequence[Any], t0_frame: int
                               ) -> List[Tuple[int, int, float, float]]:
        """Shift every ``TrajectoryPoint`` frame by ``-t0_frame`` ?
``(Frame,Visibility,X,Y)`` rows.

        Visibility/X/Y are passed through untouched (the shuttle geometry is metric-
invariant under a
        pure time shift); only the absolute frame index moves toward 0, severing the match
timeline.
        """
        rows: List[Tuple[int, int, float, float]] = []
        for p in points:
            rows.append((int(p.frame) - t0_frame, 1 if p.visible else 0, float(p.x),
float(p.y)))
        rows.sort(key=lambda r: r[0])
        return rows


    def _reset_labels(gts: Sequence[Interval], t0_sec: float) -> List[Interval]:
        """Shift every label by ``-t0_sec`` (same offset as the trajectory ? metric-
invariant)."""
        return [(s - t0_sec, e - t0_sec) for s, e in gts]


    def freeze_real_fixture(
```

```
        label: str,
        trajectory_csv_path: Any,
        labels_csv_path: Any,
        *,
        license: str,
        minors_free: bool,
        owned: bool,
        fps: float,
        frame_width: float,
        fixtures_dir: Path = FIXTURES_DIR,
        slug: Optional[str] = None,
        time_origin_reset: bool = True,
        source_note: str = "",
    ) -> Dict[str, Any]:
        """Turn an OWNED, minors-free real trajectory + golden labels into a committable,
de-attributed
        Tier-0 fixture ? reusing M1's ``replay_fixture`` for the snapshot. (§4c / §5 / §7
row P1.)

        The refusal gate (M2) makes the unsafe path impossible to reach silently: a fixture
is emitted
        ONLY when ``minors_free is True`` AND ``owned is True`` AND ``license`` is a non-
blank string.
        The committed bytes carry NO filename / video_id / match / player / capture-date ?
only an
        opaque random surrogate slug, the shuttle-only trajectory (time-origin reset by
default), the
        reset labels, and a ``kind="cleared_real"`` provenance block. A positive privacy
assertion
        re-reads the written ``expected.json`` + ``provenance.json`` and refuses if anything
attributable
        slipped in.

        NOTE (§6 honest limits): the random surrogate + reset timeline make the fixture non-
attributable
        only *relative to the unpublished source* ? it is NOT absolutely anonymous (EDPS v.
SRB). De-
        attribution does not cure provenance/licensing; this sits BEHIND counsel sign-off
(P2) and the
        owner-asserted Fork-A / minors / biometric-exclusion flags.

        Returns the recomputed ``expected.json`` snapshot dict (identical to
``replay_fixture``).
        """
        # --- (a) REFUSAL GATE (M2) ? the code-enforced provenance hard-fail (§4c / §7 ?)
-------
        reasons: List[str] = []
        if minors_free is not True:
            reasons.append("minors_free must be True (DPDP child = under 18; behavioural
monitoring "
                           "of a minor is prohibited ? §3, guardrail 'exclude minors')")
        if owned is not True:
            reasons.append("owned must be True (only owner-recorded Fork-A source is safe to
commit; "
                           "broadcast/scraped = Fork B, do NOT commit ? §3 D1)")
        if not (isinstance(license, str) and license.strip()):
            reasons.append("license must be a non-blank string (per-record provenance tag ?
§4c "
                           "'provenance hard-fail gate')")
        if reasons:
            raise RefusalError(
                "REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution
does NOT "
                "cure provenance/licensing; this gate sits behind counsel sign-off,
§6/P2):\n  - "
```

```
605                 + "\n  - ".join(reasons))
606
607         traj_path = Path(str(trajectory_csv_path))
608         labels_path = Path(str(labels_csv_path))
609
610         # --- (b) opaque RANDOM surrogate slug ? never the video_id MD5, never the source
name ---
611         out_slug = slug if slug is not None else _new_surrogate_slug()
612
613         # --- (c) read via the SHIPPED readers; labels via the STRICT loader (never lenient)
----
614         points = TrackNetRunner.parse_trajectory_csv(str(traj_path))
615         if not points:
616             raise ValueError(f"{traj_path}: no parseable trajectory rows
(Frame,Visibility,X,Y)")
617         gts = load_gt_intervals(str(labels_path), strict=True)
618
619         # --- (d) consistent time-origin reset (default ON; MUST be metric-invariant, §4c)
------
620         # Subtract the SAME offset from BOTH the trajectory frames and the label seconds:
because
621         # metrics.score is built from pure differences (interval_iou, |start-start|, |end-
end|), the
622         # score is bit-identical with vs without reset ? only the absolute timeline is
severed. The
623         # quality-floor / characterization snapshot is therefore unchanged (asserted in the
tests).
624         if time_origin_reset:
625             t0_frame = min(int(p.frame) for p in points)
626             t0_sec = t0_frame / float(fps)
627         else:
628             t0_frame, t0_sec = 0, 0.0
629         rows = _reset_trajectory_rows(points, t0_frame)
630         reset_gts = _reset_labels(gts, t0_sec)
631
632         # --- (e) write the de-attributed fixture (REUSE M1 writers + replay_fixture)
-----------
633         fdir = fixtures_dir / out_slug
634         fdir.mkdir(parents=True, exist_ok=True)
635         write_trajectory_csv(rows, fdir / "trajectory.csv")
636         write_labels_csv(reset_gts, fdir / "labels.csv")
637
638         iou = 0.5
639         prov = {
640             "slug": out_slug,
641             "schema_version": CURRENT_SCHEMA,
642             "kind": "cleared_real",
643             "label": str(label),
644             "license": license,
645             "minors_free": True,
646             "owned": True,
647             "fps": _round(fps),
648             "frame_width": _round(frame_width),
649             "iou": _round(iou),
650             "time_origin_reset": bool(time_origin_reset),
651             "windowing": {
652                 "vt": PINNED_VT,
653                 "mg": PINNED_MERGE_GAP,
654                 "mwd": PINNED_MIN_WINDOW,
655                 "max_win": PINNED_MAX_WINDOW,
656             },
657             "paths": {
658                 "trajectory": "trajectory.csv",
659                 "labels": "labels.csv",
660                 "expected": "expected.json",
```

```
            },
            "gt_hash": gt_hash(reset_gts),
            "source_note": str(source_note),
            "note": (
                "De-attributed REAL Fork-A fixture (owner-recorded, owned, minors-excluded,
biometric-"
                "free, shuttle-only). Opaque RANDOM surrogate slug; absolute timeline reset.
NON-"
                "attributable only relative to the UNPUBLISHED source (EDPS v. SRB) ? never
'anonymous'. "
                "Tier-0 / CONTROL-only. §6: behaviour-locking, not correctness; sits behind
counsel (P2)."
            ),
        }
        _write_json(fdir / "provenance.json", prov)

        expected = replay_fixture(fdir)          # REUSE M1 ? correct-by-construction
snapshot
        _write_json(fdir / "expected.json", expected)

        # --- (f) POSITIVE privacy assertion AFTER writing (scan the written text, §4c / ?)
-----
        source_paths = [str(traj_path), str(labels_path)]
        _scan_forbidden((fdir / "expected.json").read_text(encoding="utf-8"),
                        where=f"{out_slug}/expected.json", source_paths=source_paths)
        _scan_forbidden((fdir / "provenance.json").read_text(encoding="utf-8"),
                        where=f"{out_slug}/provenance.json", source_paths=source_paths)

        # --- (g) append to the manifest with kind="cleared_real" (don't clobber synthetic)
-----
        _append_manifest_entry(prov, fixtures_dir)
        return expected


    def _manifest_entry_for(prov: Dict[str, Any]) -> Dict[str, Any]:
        """A manifest row for a single fixture, derived from its provenance block."""
        slug = prov["slug"]
        return {
            "slug": slug,
            "fps": _round(float(prov["fps"])),
            "frame_width": _round(float(prov["frame_width"])),
            "windowing": dict(prov.get("windowing", {})),
            "kind": prov.get("kind", "synthetic"),
            "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),
            "paths": {
                "dir": slug,
                "trajectory": f"{slug}/trajectory.csv",
                "labels": f"{slug}/labels.csv",
                "expected": f"{slug}/expected.json",
                "provenance": f"{slug}/provenance.json",
            },
        }


    def _append_manifest_entry(prov: Dict[str, Any], fixtures_dir: Path) -> None:
        """Add/replace one fixture's manifest row WITHOUT clobbering the others (e.g.
synthetic)."""
        entries = load_manifest(fixtures_dir)
        entries = [e for e in entries if e.get("slug") != prov["slug"]]
        entries.append(_manifest_entry_for(prov))
        entries.sort(key=lambda e: str(e.get("slug")))
        _write_json(fixtures_dir / "manifest.json", entries)


    # ============================================================================
```

```
    # Manifest + loaders
    # ============================================================================
    def _write_manifest(specs: Sequence[FixtureSpec], fixtures_dir: Path) -> None:
        entries = [{
            "slug": spec.slug,
            "fps": _round(spec.fps),
            "frame_width": _round(spec.frame_width),
            "windowing": {
                "vt": PINNED_VT, "mg": PINNED_MERGE_GAP,
                "mwd": PINNED_MIN_WINDOW, "max_win": PINNED_MAX_WINDOW,
            },
            "kind": "synthetic",
            "schema_version": CURRENT_SCHEMA,
            "paths": {
                "dir": spec.slug,
                "trajectory": f"{spec.slug}/trajectory.csv",
                "labels": f"{spec.slug}/labels.csv",
                "expected": f"{spec.slug}/expected.json",
                "provenance": f"{spec.slug}/provenance.json",
            },
        } for spec in specs]
        _write_json(fixtures_dir / "manifest.json", entries)


    def load_manifest(fixtures_dir: Path = FIXTURES_DIR) -> List[Dict[str, Any]]:
        """Load the fixtures manifest (returns [] if absent)."""
        path = fixtures_dir / "manifest.json"
        if not path.is_file():
            return []
        data = _read_json(path)
        if not isinstance(data, list):  # pragma: no cover - corrupt manifest
            raise ValueError(f"{path}: manifest must be a JSON array")
        return data


    def load_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR) -> Dict[str, Any]:
        """Load one fixture's committed artifacts: ``{provenance, expected, dir}``. Refuses
a fixture
        whose ``schema_version`` exceeds ``CURRENT_SCHEMA`` (forward-incompatible)."""
        fdir = fixtures_dir / slug
        prov = _read_json(fdir / "provenance.json")
        expected = _read_json(fdir / "expected.json")
        sv = int(expected.get("schema_version", prov.get("schema_version", 0)))
        if sv > CURRENT_SCHEMA:
            raise ValueError(
                f"{slug}: schema_version {sv} > CURRENT_SCHEMA {CURRENT_SCHEMA} ? upgrade
the harness")
        return {"slug": slug, "dir": fdir, "provenance": prov, "expected": expected}


    # ============================================================================
    # Parse-compare (NEVER byte-compare): exact ints/structure, abs-tol floats
    # ============================================================================


    def compare_expected(recomputed: Dict[str, Any], committed: Dict[str, Any]) ->
List[str]:
        """Return a list of human-readable mismatch messages (empty == match).

        Parse-compare per HARD CONSTRAINT 2: structural/int fields (candidate count,
net_crossings,
        TP/FP/FN, segment count, num_pred/num_gt, slug, schema_version, gt_hash) must be
EXACTLY
```

```
        equal; float fields compared with abs tolerance 1e-6. Never a byte compare ? CRLF/float-format
        safe (the repo runs with core.autocrlf=true)."""
        out: List[str] = []

        def exact(path: str, a: Any, b: Any) -> None:
            if a != b:
                out.append(f"{path}: {a!r} != {b!r} (exact)")

        def approx(path: str, a: Any, b: Any) -> None:
            try:
                if abs(float(a) - float(b)) > _FLOAT_TOL:
                    out.append(f"{path}: {a!r} != {b!r} (|?| > {_FLOAT_TOL})")
            except (TypeError, ValueError):
                out.append(f"{path}: non-numeric float field {a!r} vs {b!r}")

        exact("slug", recomputed.get("slug"), committed.get("slug"))
        exact("schema_version", recomputed.get("schema_version"), committed.get("schema_version"))
        exact("gt_hash", recomputed.get("gt_hash"), committed.get("gt_hash"))
        approx("iou", recomputed.get("iou"), committed.get("iou"))

        rc, cc = recomputed.get("candidates", []), committed.get("candidates", [])
        if len(rc) != len(cc):
            out.append(f"candidates: count {len(rc)} != {len(cc)} (exact)")
        else:
            for i, (r, c) in enumerate(zip(rc, cc)):
                approx(f"candidates[{i}].start", r.get("start"), c.get("start"))
                approx(f"candidates[{i}].end", r.get("end"), c.get("end"))
                rs, cs = r.get("shuttle", {}), c.get("shuttle", {})
                if set(rs) != set(cs):
                    out.append(f"candidates[{i}].shuttle: keys {sorted(rs)} != {sorted(cs)} (exact)")
                else:
                    for k in rs:
                        if k == "net_crossings":
                            exact(f"candidates[{i}].shuttle.net_crossings", rs[k], cs[k])
                        else:
                            approx(f"candidates[{i}].shuttle.{k}", rs[k], cs[k])

        rseg, cseg = recomputed.get("control_segments", []), committed.get("control_segments", [])
        if len(rseg) != len(cseg):
            out.append(f"control_segments: count {len(rseg)} != {len(cseg)} (exact)")
        else:
            for i, (r, c) in enumerate(zip(rseg, cseg)):
                approx(f"control_segments[{i}][0]", r[0], c[0])
                approx(f"control_segments[{i}][1]", r[1], c[1])

        rsc, csc = recomputed.get("score", {}), committed.get("score", {})
        int_fields = ("num_pred", "num_gt", "true_positives", "false_positives", "false_negatives")
        float_fields = ("iou_threshold", "precision", "recall", "f1", "mean_iou",
                        "mean_start_error", "mean_end_error")
        for k in int_fields:
            exact(f"score.{k}", rsc.get(k), csc.get(k))
        for k in float_fields:
            approx(f"score.{k}", rsc.get(k), csc.get(k))

        return out


    # ============================================================================
    # Small JSON / path helpers (deterministic writers: sort_keys, LF, trailing newline)
    # ============================================================================
```

```
838
839
840     def _write_json(path: Path, obj: Any) -> None:
841         text = json.dumps(obj, indent=2, sort_keys=True, ensure_ascii=False)
842         path.write_text(text + "\n", encoding="utf-8", newline="\n")
843
844
845     def _read_json(path: Path) -> Any:
846         with open(path, encoding="utf-8") as f:
847             return json.load(f)
848
849
850     def _resolve_dir(slug_or_dir: Any) -> Path:
851         if isinstance(slug_or_dir, Path):
852             return slug_or_dir
853         p = Path(str(slug_or_dir))
854         if p.is_dir():
855             return p
856         return FIXTURES_DIR / str(slug_or_dir)
857
858
859     def _fps_fw(prov: Optional[Dict[str, Any]], slug: str) -> Tuple[float, float]:
860         if prov and "fps" in prov and "frame_width" in prov:
861             return float(prov["fps"]), float(prov["frame_width"])
862         spec = _spec_by_slug().get(slug)
863         if spec is not None:
864             return spec.fps, spec.frame_width
865         raise ValueError(f"{slug}: no fps/frame_width in provenance and not a built-in
spec")
866
867
868     def _fps_fw_from_prov(fdir: Path, slug: str) -> Tuple[float, float]:
869         prov_path = fdir / "provenance.json"
870         prov = _read_json(prov_path) if prov_path.is_file() else None
871         return _fps_fw(prov, slug)
872
873
874     # ============================================================================
875     # CLI
876     # ============================================================================
877
878
879     def _cmd_check(fixtures_dir: Path) -> int:
880         manifest = load_manifest(fixtures_dir)
881         if not manifest:
882             print("[golden-fixtures] no fixtures found ? run --freeze-synthetic first",
file=sys.stderr)
883             return 1
884         failures = 0
885         for entry in manifest:
886             slug = entry["slug"]
887             committed = _read_json(fixtures_dir / slug / "expected.json")
888             recomputed = replay_fixture(fixtures_dir / slug)
889             mismatches = compare_expected(recomputed, committed)
890             if mismatches:
891                 failures += 1
892                 print(f"[FAIL] {slug}: {len(mismatches)} mismatch(es) "
893                       f"(characterization = behaviour-locking, NOT correctness ? §6):")
894                 for m in mismatches:
895                     print(f"    - {m}")
896             else:
897                 sc = recomputed["score"]
898                 print(f"[ok]   {slug}: {len(recomputed['candidates'])} candidates, "
899                       f"{len(recomputed['control_segments'])} segments, F1={sc['f1']:.6f}")
900         print(f"[golden-fixtures] {len(manifest) - failures}/{len(manifest)} fixtures match
```

```
"
901               f"(synthetic-tier = plumbing correctness, not field quality ? §6)")
902         return 0 if failures == 0 else 1
903
904
905     def main(argv: Optional[Sequence[str]] = None) -> int:
906         ap = argparse.ArgumentParser(
907             description="Golden Regression Fixtures (M1) ? synthetic, CONTROL-only replay
harness.")
908         g = ap.add_mutually_exclusive_group(required=True)
909         g.add_argument("--freeze-synthetic", action="store_true",
910                        help="generate synthetic trajectory+labels for all built-in
scenarios, then "
911                             "freeze expected.json+provenance.json + write manifest.json")
912         g.add_argument("--refresh-snapshot", action="store_true",
913                        help="re-freeze expected.json from COMMITTED trajectory+labels (no
regeneration)")
914         g.add_argument("--check", action="store_true",
915                        help="replay all fixtures + parse-compare vs committed expected.json
(exit 0/1)")
916         g.add_argument("--freeze-real", action="store_true",
917                        help="(P1) de-attribute an OWNED real trajectory+labels into a
committable Tier-0 "
918                             "fixture; REFUSED without --owned --minors-free and a non-blank
--license")
919         ap.add_argument("--slug", help="single slug for --refresh-snapshot, or the surrogate
slug for "
920                                        "--freeze-real (default: a random fx_<hex> id)")
921         ap.add_argument("--all", action="store_true", help="all slugs for --refresh-
snapshot")
922         ap.add_argument("--reason", help="recorded reason for --refresh-snapshot (review the
diff!)")
923         # --freeze-real (P1 / M2 refusal gate) options:
924         ap.add_argument("--trajectory", help="(--freeze-real) source trajectory CSV
(Frame,Visibility,X,Y)")
925         ap.add_argument("--labels", help="(--freeze-real) source golden labels CSV
(start,end)")
926         ap.add_argument("--license", dest="license_", default=None,
927                         help="(--freeze-real) the source license tag (required, non-blank)")
928         ap.add_argument("--owned", action="store_true",
929                         help="(--freeze-real) assert owner-recorded Fork-A source
(required)")
930         ap.add_argument("--minors-free", dest="minors_free", action="store_true",
931                         help="(--freeze-real) assert the source contains NO minors
(required)")
932         ap.add_argument("--fps", type=float, help="(--freeze-real) source fps")
933         ap.add_argument("--frame-width", dest="frame_width", type=float, help="(--freeze-
real) source frame width (px)")
934         ap.add_argument("--label", default="cleared_real", help="(--freeze-real) non-
identifying scenario label")
935         ap.add_argument("--source-note", dest="source_note", default="",
936                         help="(--freeze-real) non-identifying note (NEVER a path/name/id)")
937         ap.add_argument("--no-time-origin-reset", dest="time_origin_reset",
action="store_false",
938                         help="(--freeze-real) DISABLE the anti-attribution time-origin reset
(default ON)")
939         ap.set_defaults(time_origin_reset=True)
940         ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
941                         help="override the fixtures directory (default: repo
eval_baselines/fixtures)")
942         args = ap.parse_args(list(argv) if argv is not None else None)
943
944         fixtures_dir: Path = args.fixtures_dir
945
946         if args.freeze_synthetic:
```

```
947             slugs = freeze_synthetic(fixtures_dir)
948             print(f"[golden-fixtures] froze {len(slugs)} synthetic fixture(s): {',
'.join(slugs)}")
949             print(f"  -> {fixtures_dir}")
950             return 0
951
952         if args.refresh_snapshot:
953             if not args.slug and not args.all:
954                 ap.error("--refresh-snapshot requires --slug S or --all")
955             if not args.reason:
956                 print("[golden-fixtures] WARNING: no --reason given. --refresh-snapshot can
"
957                       "rubber-stamp a real regression ? review the visible diff (§6).",
file=sys.stderr)
958             targets = refresh_snapshot(slug=args.slug, all_slugs=args.all,
fixtures_dir=fixtures_dir)
959             print(f"[golden-fixtures] refreshed snapshot for: {', '.join(targets)}"
960                   + (f"  (reason: {args.reason})" if args.reason else ""))
961             return 0
962
963         if args.freeze_real:
964             missing = [n for n, v in (("--trajectory", args.trajectory),
965                                       ("--labels", args.labels),
966                                       ("--fps", args.fps),
967                                       ("--frame-width", args.frame_width)) if v is None]
968             if missing:
969                 ap.error("--freeze-real requires " + ", ".join(missing))
970             try:
971                 expected = freeze_real_fixture(
972                     args.label, args.trajectory, args.labels,
973                     license=args.license_ if args.license_ is not None else "",
974                     minors_free=args.minors_free, owned=args.owned,
975                     fps=args.fps, frame_width=args.frame_width,
976                     fixtures_dir=fixtures_dir, slug=args.slug,
977                     time_origin_reset=args.time_origin_reset, source_note=args.source_note)
978             except RefusalError as e:
979                 # The code-enforced provenance gate (§4c / §7 ?): refuse loudly, exit non-
zero.
980                 print(f"[golden-fixtures] REFUSED (--freeze-real provenance gate):\n{e}",
file=sys.stderr)
981                 return 2
982             print(f"[golden-fixtures] froze de-attributed REAL fixture '{expected['slug']}'
"
983                   f"(kind=cleared_real) -> {fixtures_dir / expected['slug']}")
984             print("  §6: behaviour-locking, NOT correctness; non-attributable only relative
to the "
985                   "UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off
(P2).")
986             return 0
987
988         # --check
989         return _cmd_check(fixtures_dir)
990
991
992     if __name__ == "__main__":
993         raise SystemExit(main())
994
```

5. assistant (2026-06-16T06:27:09.235Z)

I have the core file. Now let me read the supporting files in parallel to understand the data contracts and verify the suspicions.

6. user (2026-06-16T06:27:10.647Z)

```
"""Experiment harness ? A/B + shadow + side-by-side for rally detection.

This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
no-regression gate, the pinnable classifier artifact, the persisted-candidate
substrate) already exists ? this module composes it. **No new scoring math lives
here**; everything defers to:

  - `backend.eval.metrics`        ? `score`, `match_intervals`, `interval_iou`
  - `backend.eval.training`       ? `label_candidates` (y by the same IoU matcher)
  - `backend.eval.classifier`     ? `LogisticRegression` (the pinnable JSON model)
  - `backend.eval.gemini_refine`  ? `merge_overlaps` (the blessed window merge)
  - `backend.eval.regression`     ? `gt_hash` (label-set fingerprint)
  - `backend.eval.calibration`    ? `pct_improvement`

The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
(per-frame signals ? run once on the GPU/WSL box, persisted into
`candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
given persisted candidate features it produces `List[Interval]` deterministically,
with no GPU and zero production risk. So most experiments are pure offline
re-scoring of stored data and never touch the served pipeline.

Three modes (`EXPERIMENT_HARNESS.md` §5):
  - `compare()`           ? labeled A/B vs golden labels: per-video + LOVO-honest
    aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
    held-out loop, but variant-parameterised.
  - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
    variants agree / diverge (A-only, B-only, agreed). The divergences are what a
    human spot-checks (and what two highlight reels would show side by side).
  - the promotion gate stays `run_regression.py` + `regression_baseline.json`
    (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**

Pure + offline + unit-tested. The only DB-touching helper is
`load_video_candidates`, which reads persisted candidates via
`Database.query_candidate_segments`.
"""
from __future__ import annotations

import json
import logging
import os
from dataclasses import dataclass, field
from datetime import datetime, timezone
from hashlib import sha1
from typing import Any, Callable, Dict, List, Optional, Sequence

from backend.eval.calibration import pct_improvement
from backend.eval.classifier import LogisticRegression
from backend.eval.gemini_refine import merge_overlaps
from backend.eval.metrics import Interval, match_intervals, score
from backend.eval.regression import gt_hash
from backend.eval.training import label_candidates
from backend.eval import eval_stats as _stats           # C1: CIs + paired sign test
from backend.eval import segmentation_metrics as _segm   # C4: F1@k + segment-count
ratio
from backend.eval.score_calibration import fit_isotonic, fit_platt   # C2: calibrate cue
scores

logger = logging.getLogger(__name__)

# Where a comparison run appends one reproducible record. Tracked location (NOT under
# the gitignored output/) so the leaderboard survives a clean checkout, mirroring
# regression.DEFAULT_BASELINE_PATH.
DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
```

```
    _METRICS = ("precision", "recall", "f1", "mean_iou")


    class ExperimentError(ValueError):
        """A variant cannot be evaluated as configured (e.g. a learned variant asked to
        score an unlabeled video with no pinned model, or a gate referencing an absent
        feature)."""


    # ---------------------------------------------------------------------- Variant


    @dataclass
    class Variant:
        """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).

        A variant turns one video's persisted candidate windows + features into rally
        intervals. Every knob defaults to the **control** behaviour (no gate, no learned
        filter), so a variant that merely *carries* a new, disabled cue is byte-identical
        to control ? the core no-regression guarantee.

        Fields:
          - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
            for the leaderboard and for selecting the served path (the existing
            `segmenter_registry` + `IndexingConfig` do the actual selection in production).
            New cues are recorded here default-OFF.
          - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
            ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
            `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-
shuttle
            windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
          - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
            whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
            cue persists that feature).
          - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
            `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
            required for `compare_unlabeled` on a learned variant.
          - ``feature_names`` ? the persisted features the learned filter consumes. When set
            WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
            (honest) ? the recipe, not a pinned artifact.
          - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
            overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
        """

        name: str
        segmenter: str = "wasb_hybrid"
        config_overrides: Dict[str, Any] = field(default_factory=dict)
        cue_flags: Dict[str, bool] = field(default_factory=dict)
        min_crossings: int = 0
        min_players: int = 0
        classifier_model: Optional[str] = None
        feature_names: Optional[List[str]] = None
        threshold: float = 0.5
        merge_gap: float = 1.0
        # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
        # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
        # failure where a fixed 0.5 zeroed out a small-candidate video.
        tune_threshold: bool = False          # pick the F1-best threshold on the train fold
        calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate
proba first

        def is_learned(self) -> bool:
            """True if this variant applies a learned classifier (pinned model or
```

```
recipe)."""
            return self.classifier_model is not None or bool(self.feature_names)

        def to_dict(self) -> dict:
            return {
                "name": self.name,
                "segmenter": self.segmenter,
                "config_overrides": self.config_overrides,
                "cue_flags": self.cue_flags,
                "min_crossings": self.min_crossings,
                "min_players": self.min_players,
                "classifier_model": self.classifier_model,
                "feature_names": self.feature_names,
                "threshold": self.threshold,
                "merge_gap": self.merge_gap,
                "tune_threshold": self.tune_threshold,
                "calibrate": self.calibrate,
            }

        @classmethod
        def from_dict(cls, d: dict) -> "Variant":
            known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
            return cls(**{k: v for k, v in d.items() if k in known})

        def spec_hash(self) -> str:
            """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
            and makes a result reproducible. The pinned **control** variant always hashes
the
            same, so a regression is always traceable to a recipe change, never to drift."""
            blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
            return sha1(blob.encode()).hexdigest()[:12]


    #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
    #: filter. Always the comparison reference; "rollback" = make this the served variant.
    CONTROL = Variant(name="control")


    # ---------------------------------------------------------------- persisted candidates


    @dataclass
    class VideoCandidates:
        """One video's persisted detection, ready for offline re-scoring.

        ``intervals`` are the candidate windows; ``features`` is column-major
        (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
        ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
        with `load_video_candidates`, or directly in tests.
        """

        name: str
        intervals: List[Interval]
        features: Dict[str, List[float]] = field(default_factory=dict)
        gts: Optional[List[Interval]] = None


    def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
        """Persisted feature column, defaulting missing/short columns to 0.0 (matches
        `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
        col = vc.features.get(name)
        n = len(vc.intervals)
        if col is None:
            logger.warning("variant references feature %r absent from %s ? treating as 0.0",
                           name, vc.name)
```

```
            return [0.0] * n
        if len(col) != n:
            raise ExperimentError(
                f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
        return [float(x) for x in col]


    def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
        cols = [_feature_column(vc, name) for name in feature_names]
        return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]


    def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
        """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
players)."""
        n = len(vc.intervals)
        mask = [True] * n
        if variant.min_crossings > 0:
            col = _feature_column(vc, "net_crossings")
            mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
        if variant.min_players > 0:
            col = _feature_column(vc, "players_in_court")
            mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
        return mask


    def predict(variant: Variant, vc: VideoCandidates,
                model: Optional[LogisticRegression] = None,
                threshold: Optional[float] = None,
                calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
        """Variant prediction: gate by persisted features, optionally filter by a learned
        model, then merge. Pure and deterministic.

        ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
        fold; if omitted and the variant pins ``classifier_model``, that frozen model is
        loaded. A learned variant with neither a supplied nor a pinned model raises ? there
        is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
        fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
them.
        """
        mask = _gate_mask(variant, vc)
        if variant.feature_names:
            if model is None:
                if variant.classifier_model is None:
                    raise ExperimentError(
                        f"variant {variant.name!r} is learned but has no model to predict "
                        f"with (pass a fitted model or set classifier_model)")
                model = LogisticRegression.load(variant.classifier_model)
            proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
variant.feature_names))]
            if calibrator is not None:
                proba = [calibrator(p) for p in proba]
            thr = variant.threshold if threshold is None else threshold
            mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
        kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
        return merge_overlaps(kept, gap_tol=variant.merge_gap)


    def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
                            train_videos: Sequence[VideoCandidates], iou: float
                            ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
        """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
        (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
```

```
        (use the variant default). Honest: never looks at the held-out video.
        """
        calibrator: Optional[Callable[[float], float]] = None
        if variant.calibrate:
            raw: List[float] = []
            y: List[int] = []
            for vc in train_videos:
                raw.extend(float(p) for p in
                           model.predict_proba(_feature_matrix(vc, variant.feature_names or
[])))
                y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
            if variant.calibrate == "platt":
                calibrator = fit_platt(raw, y)
            elif variant.calibrate == "isotonic":
                calibrator = fit_isotonic(raw, y)
            else:
                raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
        threshold: Optional[float] = None
        if variant.tune_threshold:
            best_t, best_f1 = variant.threshold, -1.0
            for k in range(1, 20):                       # grid 0.05..0.95 on the train
fold's rally F1
                t = round(0.05 * k, 2)
                f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
                             vc.gts or [], iou).f1 for vc in train_videos]
                mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
                if mean_f1 > best_f1:
                    best_f1, best_t = mean_f1, t
            threshold = best_t
        return calibrator, threshold


    # ------------------------------------------------------------------- variant scoring


    def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
        """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
        evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`."""
        X: List[List[float]] = []
        y: List[int] = []
        for vc in videos:
            if vc.gts is None:
                raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
            X.extend(_feature_matrix(vc, variant.feature_names or []))
            y.extend(label_candidates(vc.intervals, vc.gts, iou))
        if not X:
            raise ExperimentError("cannot train a learned variant on zero candidates")
        return LogisticRegression().fit(X, y, variant.feature_names)


    def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
        """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
        mean aggregate.

        Prediction policy:
          - **static variant** (no learned filter): predict each video directly ? no
training,
            so no leakage; per-video scoring is already honest.
          - **learned, pinned model** (``classifier_model`` set): score every video with the
            SHIPPED model. ? optimistic if those videos were in its training set ? use this
```

```
to
306             report "how the shipped artifact does here", not for the promotion decision.
307           - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
?
308             train on the OTHER videos, predict the held-out one. The number to trust.
309           - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
310         """
311         for vc in videos:
312             if vc.gts is None:
313                 raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
314
315         per_video: List[Dict[str, Any]] = []
316         predictions: Dict[str, List[Interval]] = {}
317
318         recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
319         pinned_model = (LogisticRegression.load(variant.classifier_model)
320                         if variant.classifier_model is not None else None)
321         all_model = None
322         if recipe_lovo and not honest:
323             all_model = _train(variant, videos, iou)
324
325         for i, vc in enumerate(videos):
326             cal: Optional[Callable[[float], float]] = None
327             thr: Optional[float] = None
328             if recipe_lovo and honest:
329                 others = [v for j, v in enumerate(videos) if j != i]
330                 if not others:
331                     raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
332                 model: Optional[LogisticRegression] = _train(variant, others, iou)
333                 if variant.tune_threshold or variant.calibrate:    # operating point from
the train fold
334                     assert model is not None                       # _train never returns
None
335                     cal, thr = _calibrate_and_tune(variant, model, others, iou)
336             elif recipe_lovo:                          # honest=False ? one model over all
videos
337                 model = all_model
338             else:                                      # static variant or pinned model
339                 model = pinned_model
340             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341             assert vc.gts is not None                  # guarded at function top
342             sc = score(preds, vc.gts, iou)
343             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344             predictions[vc.name] = preds
345
346         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
347                      for m in _METRICS}
348         return {
349             "variant": variant.name,
350             "spec_hash": variant.spec_hash(),
351             "is_learned": variant.is_learned(),
352             "honest_lovo": recipe_lovo and honest,
353             "per_video": per_video,
354             "aggregate": aggregate,
355             "predictions": predictions,
356         }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
```

```
mean)."""
        return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}


    def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
        """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
        gts_by_name = {v.name: (v.gts or []) for v in videos}
        overlaps = _segm.DEFAULT_OVERLAPS
        f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
        ratios: List[float] = []
        for name, preds in eval_v.get("predictions", {}).items():
            gts = gts_by_name.get(name, [])
            got = _segm.f1_at_overlaps(preds, gts, overlaps)
            for ov in overlaps:
                f1k[ov].append(got[ov])
            ratios.append(_segm.segment_count_ratio(preds, gts))

        def _mean(xs: List[float]) -> float:
            return sum(xs) / len(xs) if xs else 0.0
        out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
        out["segment_count_ratio"] = _mean(ratios)
        return out


    def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
        """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).

        ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
        so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
        the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
        """
        a_f1 = [p["f1"] for p in eval_a["per_video"]]
        b_f1 = [p["f1"] for p in eval_b["per_video"]]
        return {
            "variant_a_ci": _ci_block(eval_a),
            "variant_b_ci": _ci_block(eval_b),
            "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
            "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
                             "variant_b": _segmentation_block(videos, eval_b)},
        }


    def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
                iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
        """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
        golden corpus and reports the **B ? A** deltas, per video and in aggregate.

        The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
        learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
        `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.

        ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
```

```
CIs, a
413         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414         existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416         bare LOVO mean at n?6 is not trustworthy on its own.
417         """
418         if len(videos) < 1:
419             raise ExperimentError("compare needs at least one labeled video")
420         eval_a = evaluate_variant(videos, variant_a, iou, honest)
421         eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425
426         by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427         per_video_delta = [
428             {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429             for p in eval_b["per_video"]
430         ]
431
432         # One fingerprint over the whole label set ? detects "the deltas were measured
against
433         # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434         all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436         result: Dict[str, Any] = {
437             "iou": iou,
438             "label_set_hash": gt_hash(all_gts),
439             "num_videos": len(videos),
440             "variant_a": eval_a,
441             "variant_b": eval_b,
442             "delta": delta,
443             "delta_pct": delta_pct,
444             "per_video_delta": per_video_delta,
445         }
446         if honest_stats:
447             result["honest"] = honest_summary(videos, eval_a, eval_b)
448         return result
449
450
451     # ------------------------------------------------------------------ SxS on unlabeled video
452
453
454     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455                           agree_iou: float = 0.5) -> Dict[str, Any]:
456         """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457
458         With no labels there is no lift to measure ? instead this surfaces where the two
459         variants **agree vs diverge**, which is exactly what a human reviews (and what two
460         highlight reels would show side by side):
461
462           - ``agreed``  ? windows both variants found (matched at ``agree_iou``);
463           - ``a_only``  ? A fired, B suppressed (B's added precision, if A was over-firing);
464           - ``b_only``  ? B fired, A did not (B's new detections ? real recall or new FPs:
465             the human / a later golden label adjudicates).
466
467         Both variants must be predictable without labels: a learned variant therefore needs
a
468         pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
469         `metrics.match_intervals` ? no new matching logic.
```

```
        """
        preds_a = predict(variant_a, video)
        preds_b = predict(variant_b, video)
        mr = match_intervals(preds_a, preds_b, agree_iou)

        agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
                  for m in mr.matches]
        agree_ious = [m.iou for m in mr.matches]
        a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
        b_only = [preds_b[i] for i in mr.unmatched_gt_indices]

        def _dur(ivs: List[Interval]) -> float:
            return sum(e - s for s, e in ivs)

        return {
            "video": video.name,
            "agree_iou": agree_iou,
            "variant_a": variant_a.name,
            "variant_b": variant_b.name,
            "num_a": len(preds_a),
            "num_b": len(preds_b),
            "num_agreed": len(agreed),
            "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
            "duration_a": _dur(preds_a),
            "duration_b": _dur(preds_b),
            "agreed": agreed,
            "a_only": a_only,
            "b_only": b_only,
        }


    # ---------------------------------------------------------------------- leaderboard / DB


    def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
                          timestamp: Optional[str] = None) -> Dict[str, Any]:
        """Append a compact, reproducible record of a `compare()` result to the JSON
        leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
        `regression_baseline.json`; deliberately the size of a baseline file, not a service
        (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
        """
        row: Dict[str, Any] = {
            "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
            "iou": result.get("iou"),
            "label_set_hash": result.get("label_set_hash"),
            "num_videos": result.get("num_videos"),
            "variant_a": {"name": result["variant_a"]["variant"],
                          "spec_hash": result["variant_a"]["spec_hash"],
                          "aggregate": result["variant_a"]["aggregate"]},
            "variant_b": {"name": result["variant_b"]["variant"],
                          "spec_hash": result["variant_b"]["spec_hash"],
                          "aggregate": result["variant_b"]["aggregate"]},
            "delta": result.get("delta"),
        }
        # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
    the
        # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
        honest = result.get("honest") or {}
        if honest.get("paired_delta_f1"):
            row["honest_delta_f1"] = honest["paired_delta_f1"]
        entries: List[Dict[str, Any]] = []
        if os.path.isfile(path):
            with open(path, encoding="utf-8") as f:
                entries = json.load(f)
        entries.append(row)
```

```
534         os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
535         with open(path, "w", encoding="utf-8") as f:
536             json.dump(entries, f, indent=2)
537         logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
538                     row["variant_a"]["name"], row["variant_b"]["name"],
539                     (result.get("delta") or {}).get("f1", 0.0), path)
540         return row
541
542
543     def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                               gts: Optional[List[Interval]] = None,
545                               feature_names: Sequence[str] = ()) -> VideoCandidates:
546         """Bridge persisted detection ? `VideoCandidates` via
`Database.query_candidate_segments`.
547
548         ``feature_names`` selects which persisted features to lift out of each row's
``stats``
549         JSON (``"duration"`` is derived from the window bounds). This is the only DB-
touching
550         function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
551         but works against any `Database` (including an in-memory one in tests).
552         """
553         rows = db.query_candidate_segments(video_id, detector=detector)
554         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
555         features: Dict[str, List[float]] = {}
556         for name in feature_names:
557             if name == "duration":
558                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
559             else:
560                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561         return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562
```

7. user (2026-06-16T06:27:11.127Z)

```
1       """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3       Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4       further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5       using a few resolution/fps-INVARIANT local features, so the model transfers across
6       videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8       Flow (all local at runtime):
9         WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10        filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11        active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12        -> rally windows.
13
14      Training labels come from Gemini full-context labels via the SAME IoU matching the
15      evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16      other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17      compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18      rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20      Run:
21          python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22      """
23      from __future__ import annotations
24
```

```
    import json
    import argparse
    from typing import Dict, List, Optional, Tuple


    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
    from backend.pipeline.detectors import rally_gate
    from backend.eval.calibrate_wasb import load_golden_gts
    from backend.eval.training import label_candidates
    from backend.eval.classifier import LogisticRegression
    from backend.eval.metrics import score
    from backend.eval.gemini_refine import merge_overlaps
    from backend.eval import windowing as _windowing

    Interval = Tuple[float, float]

    FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
    # Recall-oriented proposal: deliberately permissive so real rallies are among the
    # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur).
    # Single-sourced from backend.eval.windowing so eval and serve can never drift ? see
that
    # module's docstring for the eval?serve bug these named presets close. The bare tuples
are
    # preserved (byte-identical: PROPOSAL == (0.005, 1.0, 0.5), BASELINE_LOCAL == SERVED ==
    # (0.02, 2.0, 2.0)) so every existing call site / golden JSON stays valid.
    PROPOSAL = _windowing.PROPOSAL.as_tuple()
    BASELINE_LOCAL = _windowing.SERVED.as_tuple()   # the served (production) windowing, no
learning


    def build_candidates(trajectory_csv: str, fps: float, frame_width: float,
                         proposal: Tuple[float, float, float] = PROPOSAL) ->
Tuple[List[Interval], List[List[float]]]:
        """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
rows)."""
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
        vt, mg, mwd = proposal
        wins = TrackNetRunner.trajectory_to_action_windows(
            points, fps=fps, frame_width=frame_width,
            velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
        intervals = [(w["start"], w["end"]) for w in wins]
        gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0)  # for net-
crossings
        X: List[List[float]] = []
        for w, g in zip(wins, gated):
            dur = max(1e-6, w["end"] - w["start"])
            win_frames = max(1.0, dur * fps)
            X.append([
                dur,                                            # rally length (sec) ?
fps-invariant
                float(w["peak_velocity"]),                      # already normalized by
frame_width
                float(w["mean_velocity"]),
                float(w["active_frame_count"]) / win_frames,    # active-shuttle ratio ?
fps-invariant
                float(g.crossings),                             # net crossings (count) ?
the rally signal
            ])
        return intervals, X


    def baseline_windows(trajectory_csv: str, fps: float, frame_width: float) ->
List[Interval]:
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
```

```
        vt, mg, mwd = BASELINE_LOCAL
        wins = TrackNetRunner.trajectory_to_action_windows(
            points, fps=fps, frame_width=frame_width,
            velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
        return [(w["start"], w["end"]) for w in wins]


    def load_video(spec: Dict, iou: float = 0.5) -> Dict:
        fps, fw = float(spec["fps"]), float(spec["frame_width"])
        intervals, X = build_candidates(spec["trajectory"], fps, fw)
        gts = load_golden_gts(spec["labels"])
        y = label_candidates(intervals, gts, iou)
        return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
X,
                "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
fw)}


    def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
List[Interval],
                        threshold: float = 0.5) -> List[Interval]:
        if not intervals:
            return []
        proba = model.predict_proba(X)
        pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
        return merge_overlaps(pos, gap_tol=1.0)


    def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
            model_out: Optional[str] = None) -> dict:
        videos = [load_video(s, iou) for s in specs]
        print(f"=== Distilled local rally classifier vs Gemini silver-GT "
              f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
        for v in videos:
            ceil = score(v["intervals"], v["gts"], iou).recall
            print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
                  f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")

        result: dict = {"videos": [v["name"] for v in videos]}
        if len(videos) >= 2:
            print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
out) ---")
            clf, base = [], []
            for i, held in enumerate(videos):
                others = [v for j, v in enumerate(videos) if j != i]
                X = [row for v in others for row in v["X"]]
                y = [t for v in others for t in v["y"]]
                model = LogisticRegression().fit(X, y, FEATURE_NAMES)
                preds = predict_rallies(model, held["X"], held["intervals"], threshold)
                sc = score(preds, held["gts"], iou)
                bsc = score(held["baseline"], held["gts"], iou)
                clf.append(sc)
                base.append(bsc)
                print(f"  held-out {held['name']}: LEARNED P={sc.precision:.3f}
R={sc.recall:.3f} F1={sc.f1:.3f}"
                      f"  |  baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
F1={bsc.f1:.3f}")
            def mean(xs, a):
                return sum(getattr(s, a) for s in xs) / len(xs)
            print(f"  MEAN held-out: LEARNED P={mean(clf,'precision'):.3f}
R={mean(clf,'recall'):.3f} "
                  f"F1={mean(clf,'f1'):.3f}  |  baseline P={mean(base,'precision'):.3f} "
                  f"R={mean(base,'recall'):.3f} F1={mean(base,'f1'):.3f}")
            result["mean_learned_f1"] = mean(clf, "f1")
            result["mean_baseline_f1"] = mean(base, "f1")
```

```
136
137         # Final model on ALL videos (for production / running on a fresh held-out video).
138         Xall = [row for v in videos for row in v["X"]]
139         yall = [t for v in videos for t in v["y"]]
140         final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
141         assert final.w is not None  # set by fit()
142         weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
143         print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
144         if model_out:
145             final.save(model_out)
146             print(f"final model (all videos) -> {model_out}")
147             result["model_out"] = model_out
148         return result
149
150
151     def main() -> None:
152         ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
153         ap.add_argument("--manifest", required=True,
154                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
155         ap.add_argument("--iou", type=float, default=0.5)
156         ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
157         ap.add_argument("--model-out", default="output/local_rally_model.json")
158         args = ap.parse_args()
159         with open(args.manifest, encoding="utf-8") as f:
160             specs = json.load(f)
161         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
162
163
164     if __name__ == "__main__":
165         main()
166
```

8. user (2026-06-16T06:27:11.514Z)

```
1       """Pure temporal interval-matching metrics for rally evaluation.
2
3       No I/O, no DB, no global state ? every function here is deterministic and unit
4       tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5       start < end. We score a list of *predicted* rally intervals against a list of
6       *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7       """
8
9       from dataclasses import dataclass, field
10      from typing import List, Tuple, Optional
11
12      Interval = Tuple[float, float]
13
14
15      def interval_iou(a: Interval, b: Interval) -> float:
16          """Temporal intersection-over-union of two intervals.
17
18          Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19          overlap / union, both measured as durations on the timeline.
20          """
21          a_start, a_end = a
22          b_start, b_end = b
23          if a_end <= a_start or b_end <= b_start:
24              return 0.0
25          inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26          if inter <= 0.0:
27              return 0.0
28          union = (a_end - a_start) + (b_end - b_start) - inter
```

```
        return inter / union if union > 0 else 0.0


    @dataclass
    class Match:
        """One matched predicted?ground-truth pair."""
        pred_index: int
        gt_index: int
        iou: float


    @dataclass
    class MatchResult:
        """Outcome of matching predictions to ground truth at a given IoU threshold."""
        matches: List[Match] = field(default_factory=list)
        unmatched_pred_indices: List[int] = field(default_factory=list)  # false positives
        unmatched_gt_indices: List[int] = field(default_factory=list)    # false negatives
(misses)


    def match_intervals(preds: List[Interval], gts: List[Interval],
                        iou_threshold: float = 0.5) -> MatchResult:
        """Greedy one-to-one matching of predictions to ground truth by descending IoU.

        Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
        are claimed highest-IoU first, and each prediction/GT can be used once. This
        keeps a single prediction from being credited for two ground-truth rallies
        (and vice versa).
        """
        candidates = []
        for pi, p in enumerate(preds):
            for gi, g in enumerate(gts):
                iou = interval_iou(p, g)
                # Require real overlap: at iou_threshold=0.0 a plain `iou >= threshold`
would
                # match completely disjoint intervals (IoU 0.0), fabricating perfect scores.
                if iou > 0.0 and iou >= iou_threshold:
                    candidates.append((iou, pi, gi))
        # Highest IoU first; ties broken by indices for determinism.
        candidates.sort(key=lambda c: (-c[0], c[1], c[2]))

        used_pred, used_gt = set(), set()
        matches: List[Match] = []
        for iou, pi, gi in candidates:
            if pi in used_pred or gi in used_gt:
                continue
            used_pred.add(pi)
            used_gt.add(gi)
            matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))

        unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
        unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
        return MatchResult(matches=matches,
                           unmatched_pred_indices=unmatched_pred,
                           unmatched_gt_indices=unmatched_gt)


    @dataclass
    class Score:
        """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
        iou_threshold: float
        num_pred: int
        num_gt: int
        true_positives: int
        false_positives: int
```

```
92          false_negatives: int
93          precision: float
94          recall: float
95          f1: float
96          mean_iou: float            # over matched pairs (0.0 if none)
97          mean_start_error: float    # mean |pred_start - gt_start| over matches, seconds
98          mean_end_error: float      # mean |pred_end - gt_end| over matches, seconds
99
100         def as_dict(self) -> dict:
101             return self.__dict__.copy()
102
103
104     def score(preds: List[Interval], gts: List[Interval],
105               iou_threshold: float = 0.5,
106               match_result: Optional[MatchResult] = None) -> Score:
107         """Compute precision/recall/F1, mean matched IoU, and boundary errors.
108
109         Pass a precomputed `match_result` to avoid re-matching (e.g. when also
110         extracting the failure lists); otherwise matching is done here.
111         """
112         mr = match_result if match_result is not None else match_intervals(preds, gts,
iou_threshold)
113
114         tp = len(mr.matches)
115         fp = len(mr.unmatched_pred_indices)
116         fn = len(mr.unmatched_gt_indices)
117
118         precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
119         recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
120         f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
else 0.0
121
122         if mr.matches:
123             mean_iou = sum(m.iou for m in mr.matches) / tp
124             mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
mr.matches) / tp
125             mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
mr.matches) / tp
126         else:
127             mean_iou = mean_start_err = mean_end_err = 0.0
128
129         return Score(
130             iou_threshold=iou_threshold,
131             num_pred=len(preds), num_gt=len(gts),
132             true_positives=tp, false_positives=fp, false_negatives=fn,
133             precision=precision, recall=recall, f1=f1,
134             mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
135         )
136
137
138     def pr_curve(preds: List[Interval], gts: List[Interval],
139                  thresholds: Optional[List[float]] = None) -> List[Score]:
140         """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
141
142         Lets you see strict-vs-lenient behaviour rather than a single operating point.
143         """
144         if thresholds is None:
145             thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
146         return [score(preds, gts, iou_threshold=t) for t in thresholds]
147
```

9. assistant (2026-06-16T06:27:15.151Z)

Now let me read the remaining analysis files in parallel: gt_loader, regression, tracknet_runner, rally_gate, and the experiment.py imports to verify the numpy-import-on-clean-

clone suspicion.

10. user (2026-06-16T06:27:16.655Z)

```
1       """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).
2
3       The repo historically carried two incompatible loaders:
4       - ``annotations.load_annotations``: positional ``start,end`` (the collector's
5         ``curate export`` format), raises on bad rows;
6       - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
7         (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.
8
9       A file in one convention fed to the other loader yielded zero/garbage intervals with no
10      error. This module accepts BOTH conventions (header-mapped, else positional), so every
11      quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
12      flows through one reader. Both legacy entry points now delegate here.
13
14      Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
15      - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16        columns ? rally_number, ending_reason, sport, ? ? are ignored);
17      - no header: columns 0,1 positionally.
18      Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
(``annotations.parse_timestamp``).
19      """
20      import csv
21      import logging
22      from typing import List, Optional, Tuple
23
24      from backend.eval.annotations import parse_timestamp
25
26      logger = logging.getLogger(__name__)
27
28      Interval = Tuple[float, float]
29
30      _START_NAMES = ("start", "start_time")
31      _END_NAMES = ("end", "end_time")
32
33
34      def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
35          """If `row` is a header naming start/end columns, return their indices, else
None."""
36          cells = [(c or "").strip().lower() for c in row]
37          start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
38          end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
39          if start_idx is not None and end_idx is not None:
40              return start_idx, end_idx
41          return None
42
43
44      def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
45          """Load GT rally intervals from either golden-CSV convention.
46
47          strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
48          (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
49          strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
50          skipped ? a silently shrinking GT set flatters precision.
51
52          Returns intervals sorted by start; overlapping GT intervals are logged as a warning
53          (they break one-to-one matching and usually indicate a labeling error).
54          """
55          intervals: List[Interval] = []
56          skipped = 0
57          cols: Optional[Tuple[int, int]] = None   # (start_idx, end_idx); None until detected
58          saw_header = False
```

```
59
60          with open(csv_path, newline="", encoding="utf-8-sig") as f:
61              for lineno, row in enumerate(csv.reader(f), start=1):
62                  if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
63                      continue
64                  if cols is None and not saw_header:
65                      header = _header_columns(row)
66                      if header is not None:
67                          cols = header
68                          saw_header = True
69                          continue            # consume the header row
70                      cols = (0, 1)           # headerless: positional start,end
71                  assert cols is not None     # every branch above assigns it
72                  si, ei = cols
73                  try:
74                      if len(row) <= max(si, ei):
75                          raise ValueError(f"expected at least {max(si, ei) + 1} columns, got
{row!r}")
76                      start = parse_timestamp(row[si])
77                      end = parse_timestamp(row[ei])
78                      if start < 0:
79                          raise ValueError(f"negative start ({start})")
80                      if start >= end:
81                          raise ValueError(f"start ({start}) >= end ({end})")
82                  except ValueError as e:
83                      if strict:
84                          raise ValueError(f"{csv_path}:{lineno}: {e}") from e
85                      skipped += 1
86                      continue
87                  intervals.append((start, end))
88
89          if skipped:
90              logger.warning("%s: skipped %d malformed GT row(s) (strict=False)", csv_path,
skipped)
91
92          intervals.sort(key=lambda iv: iv[0])
93          for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
94              if s2 < e1:
95                  logger.warning("%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
96                                 "labeling error? Overlaps break one-to-one matching.",
97                                 csv_path, s1, e1, s2, e2)
98          return intervals
99
```

11. user (2026-06-16T06:27:16.984Z)

```
1       """Regression runner ? verify the production tool against golden-set ground truth.
2
3       Implements the explicit ask: *"given a video, get a golden set created and verify
4       its precision and recall against the production tool."* This is deliberately a thin
5       wrapper over machinery that already exists:
6
7           ground truth  ? AnnotationProvider (file / vendor / oracle)  [backend.annotations]
8           predictions   ? the production pipeline's stored DB rows     [backend.eval.harness]
9           comparison    ? interval matching + P/R/F1                   [backend.eval.metrics]
10          baseline      ? a per-video snapshot of "known good" numbers [this module]
11
12      A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13      baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14      later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15      independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16      """
17      import os
18      import json
19      import hashlib
```

```
    import logging
    from dataclasses import dataclass, asdict
    from typing import Dict, List, Optional

    from backend.infrastructure.database import Database
    from backend.eval import metrics
    from backend.eval.harness import get_predictions
    from backend.annotations import annotation_registry

    logger = logging.getLogger(__name__)

    # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
    # actually load a committed baseline to compare against.
    DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
    DEFAULT_TOLERANCE = 0.05


    def gt_hash(ground_truth: List[metrics.Interval]) -> str:
        """Stable short hash of the GT intervals ? detects a baseline taken against
different labels."""
        norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
        return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]


    @dataclass
    class RegressionResult:
        video_id: str
        stage: str
        iou_threshold: float
        precision: float
        recall: float
        f1: float
        # Baseline values compared against (None when no baseline existed yet).
        baseline_precision: Optional[float]
        baseline_recall: Optional[float]
        tolerance: float
        regressed: bool
        reasons: List[str]
        # True only when a baseline existed to compare against. Distinguishes a genuine PASS
        # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
        has_baseline: bool = False
        gt_hash: Optional[str] = None

        def as_dict(self) -> dict:
            return asdict(self)


    # ---------------------------------------------------------------- baseline store

    def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
        """Load the per-video baseline snapshot file (returns {} if absent)."""
        if not os.path.isfile(path):
            return {}
        with open(path) as f:
            return json.load(f)


    def _baseline_key(video_id: str, stage: str) -> str:
        return f"{video_id}::{stage}"


    def save_baseline(video_id: str, stage: str, precision: float, recall: float,
                      f1: float, path: str = DEFAULT_BASELINE_PATH,
                      iou_threshold: Optional[float] = None, detector: Optional[str] = None,
```

```
                        gt_fingerprint: Optional[str] = None) -> None:
        """Snapshot a video/stage's current numbers as the new 'known good' baseline.

        Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
        later run computed under different settings (or against different labels) is
detected
        as incommensurable rather than silently compared.
        """
        baselines = load_baselines(path)
        baselines[_baseline_key(video_id, stage)] = {
            "video_id": video_id, "stage": stage,
            "precision": precision, "recall": recall, "f1": f1,
            "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
        }
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
        with open(path, "w") as f:
            json.dump(baselines, f, indent=2, sort_keys=True)
        logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
_baseline_key(video_id, stage), precision, recall)


    # ---------------------------------------------------------------- the runner

    def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
                stage: str = "final", iou_threshold: float = 0.5,
                detector: Optional[str] = None,
                tolerance: float = DEFAULT_TOLERANCE,
                baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
        """Score stored predictions for one video against ground truth and compare to
baseline.

        `ground_truth` is a list of (start, end) intervals ? typically obtained from an
        AnnotationProvider (see `regress_with_provider`). A regression is flagged when
        precision OR recall falls more than `tolerance` below the snapshotted baseline.
        If no baseline exists yet, `regressed` is False (nothing to compare to) and the
        caller can choose to snapshot the current numbers via `save_baseline`.
        """
        preds = get_predictions(db, video_id, stage, detector=detector)
        s = metrics.score(preds, ground_truth, iou_threshold)
        fp = gt_hash(ground_truth)

        baselines = load_baselines(baseline_path)
        b = baselines.get(_baseline_key(video_id, stage))

        reasons: List[str] = []
        regressed = False
        has_baseline = b is not None
        base_p = base_r = None
        if b is not None:
            base_p, base_r = b["precision"], b["recall"]
            # Incommensurable comparison guard: a baseline taken at a different IoU/detector
or
            # against different labels is not a valid reference ? flag it instead of
trusting it.
            b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
b.get("gt_hash")
            if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
                regressed = True
                reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
(incommensurable)")
            if b_det is not None and b_det != detector:
                regressed = True
                reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
            if b_hash is not None and b_hash != fp:
```

```
140                 regressed = True
141                 reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
142             if s.precision < base_p - tolerance:
143                 regressed = True
144                 reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
145             if s.recall < base_r - tolerance:
146                 regressed = True
147                 reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
148         else:
149             reasons.append("no baseline recorded for this video/stage ? nothing to compare")
150 
151         return RegressionResult(
152             video_id=video_id, stage=stage, iou_threshold=iou_threshold,
153             precision=s.precision, recall=s.recall, f1=s.f1,
154             baseline_precision=base_p, baseline_recall=base_r,
155             tolerance=tolerance, regressed=regressed, reasons=reasons,
156             has_baseline=has_baseline, gt_hash=fp,
157         )
158 
159 
160     def regress_with_provider(db: Database, video_id: str, video_ref: str,
161                               tier: str = "file", stage: str = "final",
162                               iou_threshold: float = 0.5,
163                               detector: Optional[str] = None,
164                               tolerance: float = DEFAULT_TOLERANCE,
165                               baseline_path: str = DEFAULT_BASELINE_PATH,
166                               guideline: Optional[str] = None,
167                               sport: str = "badminton",
168                               provider_kwargs: Optional[dict] = None) -> RegressionResult:
169         """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.
170 
171         `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
172         once those providers land). `video_ref` is whatever that provider expects (a CSV
173         path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
174         predictions in the DB (the file MD5).
175         """
176         provider = annotation_registry.get(tier, **(provider_kwargs or {}))
177         gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
178         return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
179                        detector=detector, tolerance=tolerance, baseline_path=baseline_path)
180 
181 
182     def format_result(r: RegressionResult) -> str:
183         verdict = "REGRESSED" if r.regressed else "ok"
184         head = (f"[{verdict}] {r.video_id} ({r.stage}) "
185                 f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
186         if r.baseline_precision is not None:
187             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
188         return head + "".join(f"\n   - {why}" for why in r.reasons)
189 
```

12. user (2026-06-16T06:27:17.929Z)

```
1       """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2 
3       This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4       inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5 
6           1. Translate the Windows video path to its WSL-visible form (and optionally
7              stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
```

```
8           2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9           3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10          4. Convert that trajectory into high-motion "action windows" ? the same
11             candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12             pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex
20      import logging
21      import subprocess
22      from dataclasses import dataclass
23      from typing import List, Dict, Any, Optional, Tuple
24
25      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27      logger = logging.getLogger(__name__)
28
29
30      @dataclass
31      class TrackNetConfig:
32          """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "TrackNetV4"
37          weights_path: str = ""                          # WSL path to the .h5/.keras weights
(REQUIRED)
38          queue_length: int = 5
39          stage_in_wsl: bool = True                       # copy video into WSL fs first
(perf)
40          wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
in WSL
41          timeout_sec: int = 1800                         # 30 min; kills a hung call (0 = no
timeout)
42          keep_frames: bool = False                       # retain staged video after success
(debug only)
43          min_free_gb: float = 0.0                        # warn if WSL free space below this
(0 = off)
44
45          @classmethod
46          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47              tn = (idx_cfg or {}).get("tracknet", {}) or {}
48              return cls(
49                  distro=tn.get("wsl_distro", "Ubuntu"),
50                  repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51                  conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52                  conda_env=tn.get("conda_env", "TrackNetV4"),
53                  weights_path=tn.get("weights_path", ""),
54                  queue_length=int(tn.get("queue_length", 5)),
55                  stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56                  wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57                  timeout_sec=int(tn.get("timeout_sec", 1800)),
58                  keep_frames=bool(tn.get("keep_frames", False)),
59                  min_free_gb=float(tn.get("min_free_gb", 0.0)),
60              )
61
62
63      def to_wsl_mnt_path(win_path: str) -> str:
64          """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
```

```
65
66          Already-POSIX paths (starting with / or ~) are returned unchanged so the
67          same helper is safe to call on either kind of path.
68          """
69          p = str(win_path)
70          if p.startswith("/") or p.startswith("~"):
71              return p
72          p = p.replace("\\", "/")
73          if len(p) >= 2 and p[1] == ":":
74              drive = p[0].lower()
75              return f"/mnt/{drive}{p[2:]}"
76          return p
77
78
79      @dataclass
80      class TrajectoryPoint:
81          frame: int
82          visible: bool
83          x: float
84          y: float
85
86
87      class TrackNetRunner(WslCommandMixin, DetectorRunner):
88          """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90          Implements the common DetectorRunner contract so a future Linux-native or
91          alternative trajectory runner can be substituted without touching the hybrids.
92          """
93
94          def __init__(self, cfg: TrackNetConfig):
95              self.cfg = cfg
96
97          def healthcheck(self) -> Tuple[bool, str]:
98              """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100             Returns (ok, message). Cheap to call before a long run.
101             """
102             cmd = (
103                 f"conda activate {self.cfg.conda_env} && "
104                 f"python -c \"import tensorflow as tf; "
105                 f"print('TF', tf.__version__, 'GPUs',
len(tf.config.list_physical_devices('GPU')))\""
106             )
107             try:
108                 res = self._wsl_bash(cmd)
109             except FileNotFoundError:
110                 return False, "`wsl` not found ? is this running on Windows with WSL2
installed?"
111             except subprocess.TimeoutExpired:
112                 return False, "WSL healthcheck timed out."
113             out = (res.stdout or "").strip()
114             if res.returncode != 0:
115                 return False, f"WSL env check failed: {(res.stderr or out).strip()}"
116             return True, out
117
118         # ------------------------------------------------------------------ inference
119
120         def run_predict(self, video_win_path: str, output_win_dir: str,
121                         video_id: Optional[str] = None) -> Optional[str]:
122             """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.
123
124             ``output_win_dir`` is a Windows directory (under the repo's output/) that
125             WSL writes into via its /mnt view, so the Windows process can read the CSV
126             back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
```

```
            therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
            a filename cannot collide on the output CSV.
            """
            if not self.cfg.weights_path:
                logger.error(
                    "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
                    "in config.json to the WSL path of your trained TrackNetV4 weights."
                )
                return None

            # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
            # relative paths through unchanged, so WSL would resolve them against the
            # predict.py cwd instead of the repo (same bug class as wasb_runner).
            output_win_dir = os.path.abspath(output_win_dir)
            os.makedirs(output_win_dir, exist_ok=True)
            # Normalize separators so basename/splitext work when tests (or collector)
            # pass Windows-style paths on a Linux host.
            video_win_path = str(video_win_path).replace("\\", "/")
            base = os.path.basename(video_win_path)
            stem, ext = os.path.splitext(base)

            # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
            if ext.lower() not in (".mp4", ".avi"):
                logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
                return None

            wsl_out = to_wsl_mnt_path(output_win_dir)
            # Namespace by video_id so two same-filename videos don't collide on the CSV.
            # predict.py derives its output name from the (staged) video stem, so staging
            # under the namespaced name propagates into "<key>_predict.csv".
            key = f"{stem}__{video_id[:12]}" if video_id else stem
            expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")

            # Resolve the video path WSL will read.
            if self.cfg.stage_in_wsl:
                staged = self._stage_video(video_win_path, f"{key}{ext}")
                if staged is None:
                    return None
                wsl_video = staged
            else:
                # No staging: predict.py reads /mnt directly and writes
"<stem>_predict.csv".
                # We cannot rename its output, so fall back to the un-namespaced name.
                wsl_video = to_wsl_mnt_path(video_win_path)
                expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")

            if self.cfg.min_free_gb > 0:
                free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
                if free is not None and free < self.cfg.min_free_gb:
                    logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
                                   "consider running tools/cleanup_caches.py.",
                                   free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)

            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"cd {self.cfg.repo_dir}/src && "
                f"python predict.py "
                f"--video_path {wsl_tilde_quote(wsl_video)} "
                f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
                f"--output_dir {wsl_tilde_quote(wsl_out)} "
                f"--queue_length {self.cfg.queue_length}"
            )
            logger.info("Running TrackNet inference on %s ...", base)
            try:
                res = self._wsl_bash(cmd)
```

```
            except subprocess.TimeoutExpired:
                logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
                return None

            if res.returncode != 0:
                logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
                return None

            if not os.path.exists(expected_csv_win):
                logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
                return None
            logger.info("TrackNet trajectory CSV: %s", expected_csv_win)

            # Success ? the CSV is the durable output; drop the staged video copy (a pure,
            # regenerable intermediate). On earlier failure we return above without
cleaning.
            if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
                logger.info("Cache hygiene: removing staged video for %s "
                            "(set indexing.tracknet.keep_frames=true to retain).", key)
                self._wsl_rm_rf([wsl_video])
            return expected_csv_win

        def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
            """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).

            Returns the WSL path to the staged copy, or None on failure.
            """
            src_mnt = to_wsl_mnt_path(video_win_path)
            reason = self.wsl_source_unreadable_reason(src_mnt)
            if reason:
                logger.error("Cannot stage video into WSL: %s", reason)
                return None
            dest = f"{self.cfg.wsl_stage_dir}/{base}"
            cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("Staging video into WSL timed out.")
                return None
            if res.returncode != 0 or "OK" not in (res.stdout or ""):
                logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
                return None
            return dest

        # ------------------------------------------------------------- CSV -> windows

        @staticmethod
        def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
            """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
            points: List[TrajectoryPoint] = []
            with open(csv_win_path, newline="") as f:
                reader = csv.DictReader(f)
                for row in reader:
                    try:
                        points.append(TrajectoryPoint(
                            frame=int(row["Frame"]),
                            visible=int(row["Visibility"]) == 1,
                            x=float(row["X"]),
                            y=float(row["Y"]),
                        ))
```

```
                    except (KeyError, ValueError):
                        continue
            points.sort(key=lambda p: p.frame)
            return points

        @staticmethod
        def trajectory_to_action_windows(
            points: List[TrajectoryPoint],
            fps: float,
            frame_width: float,
            chunk_start: float = 0.0,
            velocity_thresh: float = 0.02,
            merge_gap: float = 2.0,
            min_window_duration: float = 2.0,
            chunk_index: Optional[int] = None,
            max_window_duration: float = 0.0,
            min_split_gap: float = 0.5,
        ) -> List[Dict[str, Any]]:
            """Convert a shuttle trajectory into candidate rally windows.

            A frame is "active" when the shuttle is visible AND its inter-frame
            displacement (normalised by frame width, per second) exceeds
            ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
            ``merge_gap`` seconds of each other are grouped into one window; short
            windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
            shape feeding the AI-handoff phase is identical.

            ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
            the *over-merge* failure where "shuttle moving" is conflated with "rally in
progress"
            and a single window swallows several rallies + the dead time between them. When
set, any
            window longer than this is recursively SPLIT at its largest internal inactivity
gap (the
            longest stretch with no in-flight shuttle ? the most likely rally boundary),
provided
            that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
fix from
            ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
recall cannot
            drop, and it is config-gated default-OFF until measured on the golden set.
            """
            if fps <= 0:
                fps = 30.0
            if frame_width <= 0:
                frame_width = 1280.0

            # Compute per-frame normalised velocity from the last *visible* point.
            active = []  # (frame_idx, velocity)
            prev: Optional[TrajectoryPoint] = None
            for p in points:
                if not p.visible:
                    prev = None  # break the trajectory; absent shuttle = not in flight
                    continue
                if prev is not None:
                    dframes = p.frame - prev.frame
                    if dframes > 0:
                        dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
                        velocity = (dist / frame_width) * (fps / dframes)
                        if velocity > velocity_thresh:
                            active.append((p.frame, velocity))
                prev = p

            if not active:
```

```
                return []

            max_gap_frames = int(merge_gap * fps)

            def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
                vels = [v for _, v in group]
                return {
                    "start": group[0][0] / fps + chunk_start,
                    "end": group[-1][0] / fps + chunk_start,
                    "active_frame_count": len(group),
                    "peak_velocity": max(vels),
                    "mean_velocity": sum(vels) / len(vels),
                    "track_id_count": 1,  # single shuttle; kept for window-shape parity
                    "chunk_index": chunk_index,
                }

            groups: List[List[Tuple[int, float]]] = []
            group = [active[0]]
            for af in active[1:]:
                if af[0] - group[-1][0] <= max_gap_frames:
                    group.append(af)
                else:
                    groups.append(group)
                    group = [af]
            groups.append(group)

            if max_window_duration > 0:
                groups = TrackNetRunner._split_overlong_groups(
                    groups, int(max_window_duration * fps), int(min_split_gap * fps))

            windows = [_finalize(g) for g in groups]
            return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

        @staticmethod
        def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
int,
                                   min_split_frames: int) -> List[List[Tuple[int, float]]]:
            """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
            internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
            boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
            invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
            rally). Iterative worklist to bound stack depth."""
            if max_win_frames <= 0:
                return groups
            out: List[List[Tuple[int, float]]] = []
            work = list(groups)
            while work:
                g = work.pop()
                if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
                    out.append(g)
                    continue
                best_i, best_gap = -1, -1
                for i in range(1, len(g)):
                    gap = g[i][0] - g[i - 1][0]
                    if gap > best_gap:
                        best_gap, best_i = gap, i
                if best_i < 0 or best_gap < min_split_frames:
                    out.append(g)  # too long but no real dead-time gap ? keep as one rally
                    continue
                work.append(g[:best_i])
                work.append(g[best_i:])
```

```
            out.sort(key=lambda grp: grp[0][0])
            return out

```

13. user (2026-06-16T06:27:18.568Z)

```
    """Rally-start gate: suppress between-point false fires using shuttle net-crossings.

    WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
    the opponent for service after losing a point, that single trajectory looks like a
    rally. The discriminator is structural, not duration-based: a real rally has the
    shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
    single service feed crosses ~once.

    This module is camera-agnostic and needs NO new model ? it works on the trajectory
    we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
    axis with the larger shuttle spread) and the net position (median shuttle coord on
    that axis ? the shuttle clusters at the two ends and crosses the net region often,
    so the median sits near the net), then counts sign changes of (coord - net) across
    the visible points inside a candidate window, with a deadband to ignore jitter.

    Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
    Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
    windows from trajectory_to_action_windows, or to fold into it later.
    """
    from __future__ import annotations

    from dataclasses import dataclass
    from typing import List, Optional, Sequence, Tuple

    Interval = Tuple[float, float]


    @dataclass
    class GatedWindow:
        start: float
        end: float
        crossings: int
        visible_points: int
        kept: bool


    def _spread(vals: Sequence[float]) -> float:
        """Robust spread = p95 - p5 (ignores a few outlier detections)."""
        if len(vals) < 2:
            return 0.0
        s = sorted(vals)
        lo = s[max(0, int(0.05 * (len(s) - 1)))]
        hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
        return hi - lo


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])


    def estimate_play_axis(points) -> Tuple[str, float, float]:
        """Return (axis 'x'|'y', net_value, span) from visible points.

        Play axis = the image axis along which the shuttle travels most (larger spread).
        For a baseline camera that's Y (players top/bottom); for a side camera, X.
```

```
        """
        xs = [p.x for p in points if p.visible]
        ys = [p.y for p in points if p.visible]
        sx, sy = _spread(xs), _spread(ys)
        if sx >= sy:
            return "x", _median(xs), sx
        return "y", _median(ys), sy


    def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
        """Count sign changes of (coord - net) across visible points, ignoring the
        deadband zone near the net (so jitter at the net doesn't inflate the count)."""
        crossings = 0
        last_side = None
        for p in window_points:
            if not p.visible:
                continue
            v = (p.x if axis == "x" else p.y) - net
            if abs(v) < deadband:
                continue
            side = v > 0
            if last_side is not None and side != last_side:
                crossings += 1
            last_side = side
        return crossings


    def _points_in(points, fps: float, start: float, end: float):
        return [p for p in points if start * fps <= p.frame <= end * fps]


    def gate_windows(points, windows: List[Interval], fps: float,
                     min_crossings: int = 2, deadband_frac: float = 0.06,
                     estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
        """Annotate each (start,end) window with its shuttle net-crossing count and a
        keep decision (crossings >= min_crossings). Deadband is a fraction of the play
        axis span. Net/axis are estimated once over the whole trajectory (stable).

        `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
        `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't
redone
        per window ? same result, avoids the per-window re-estimation."""
        axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
        deadband = deadband_frac * span
        out: List[GatedWindow] = []
        for (s, e) in windows:
            wp = _points_in(points, fps, s, e)
            c = count_net_crossings(wp, axis, net, deadband)
            vis = sum(1 for p in wp if p.visible)
            out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
                                   kept=c >= min_crossings))
        return out


    def apply_gate(points, windows: List[Interval], fps: float,
                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
        """Convenience: return only the windows that pass the crossing gate."""
        return [(g.start, g.end) for g in
                gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]

```

14. assistant (2026-06-16T06:27:22.306Z)

Now let me read the test files, the CI workflow, the fixtures, and check the experiment.py

imports for the numpy-on-clean-clone suspicion (classifier, calibration, eval_stats, etc.).

15. user (2026-06-16T06:27:23.273Z)

```
1       """Golden Regression Fixtures (M1) ? characterization tests over the committed synthetic
Tier-0.
2
3       Parametrized over ``load_manifest()`` so every committed fixture is exercised. The gate
is
4       **parse-compare** (never byte-compare), **CONTROL-only** (pure stdlib, no numpy on the
replay
5       path), and asserts the Tier-0 privacy invariant positively (no
person/audio/pose/gait/face keys).
6
7       §6 honest-limits caveat: green here proves only that the deterministic post-trajectory
transforms
8       are unchanged for the frozen synthetic cases ? characterization is behaviour-locking,
NOT
9       correctness, and synthetic-tier numbers measure plumbing, not field quality.
10      """
11      import json
12
13      import pytest
14
15      from backend.eval import golden_fixtures as gf
16
17      # Slugs discovered from the committed manifest ? the suite grows by accretion
automatically.
18      MANIFEST = gf.load_manifest()
19      SLUGS = [entry["slug"] for entry in MANIFEST]
20
21
22      def test_manifest_is_non_empty():
23          """A committed manifest with at least one fixture must exist (else nothing is
gated)."""
24          assert SLUGS, "no fixtures in eval_baselines/fixtures/manifest.json ? run --freeze-
synthetic"
25
26
27      @pytest.mark.parametrize("slug", SLUGS)
28      def test_replay_matches_committed_expected(slug):
29          """Replay the fixture and parse-compare against its committed expected.json.
30
31          Characterization = behaviour-locking, NOT correctness (§6): a non-empty mismatch
list means
32          the deterministic post-trajectory behaviour CHANGED for this frozen case, not that
it is wrong.
33          """
34          committed = gf.load_fixture(slug)["expected"]
35          recomputed = gf.replay_fixture(slug)
36          mismatches = gf.compare_expected(recomputed, committed)
37          assert mismatches == [], (
38              f"{slug}: behaviour changed vs frozen snapshot (characterization, not
correctness ? §6):\n"
39              + "\n".join(f"  - {m}" for m in mismatches)
40          )
41
42
43      @pytest.mark.parametrize("slug", SLUGS)
44      def test_schema_version_le_current(slug):
45          """Every committed fixture's schema_version must be <= CURRENT_SCHEMA (forward-
compatible)."""
46          fx = gf.load_fixture(slug)
47          assert int(fx["expected"]["schema_version"]) <= gf.CURRENT_SCHEMA
48          assert int(fx["provenance"]["schema_version"]) <= gf.CURRENT_SCHEMA
```

```
49
50
51      @pytest.mark.parametrize("slug", SLUGS)
52      def test_no_person_or_audio_keys(slug):
53          """Positive privacy assertion (D3 / §4c): NO biometric/identity/non-shuttle key
appears
54          anywhere in the committed expected.json ? not trusting a silent 0.0 default."""
55          fdir = gf.FIXTURES_DIR / slug
56          raw = (fdir / "expected.json").read_text(encoding="utf-8").lower()
57          for forbidden in gf.FORBIDDEN_KEYS:
58              assert forbidden not in raw, (
59                  f"{slug}: forbidden key substring {forbidden!r} present in expected.json
(Tier-0 leak)")
60
61          expected = gf.load_fixture(slug)["expected"]
62          for i, cand in enumerate(expected["candidates"]):
63              # The only feature block is the shuttle block, and it holds EXACTLY the 5
shuttle cols.
64              assert set(cand.keys()) == {"start", "end", "shuttle"}, f"{slug}: candidate[{i}]
has extra keys"
65              assert set(cand["shuttle"].keys()) == set(gf.SHUTTLE_FEATURES), (
66                  f"{slug}: candidate[{i}].shuttle is not exactly the 5 shuttle features")
67
68
69      @pytest.mark.parametrize("slug", SLUGS)
70      def test_synthetic_provenance_tags(slug):
71          """Tier-0 hard tags: kind=synthetic, license=synthetic, minors_free=true (constraint
5)."""
72          prov = gf.load_fixture(slug)["provenance"]
73          assert prov["kind"] == "synthetic"
74          assert prov["license"] == "synthetic"
75          assert prov["minors_free"] is True
76          # Windowing pinned in the fixture (not the named SERVED constant) ? constraint 3.
77          assert set(prov["windowing"]) == {"vt", "mg", "mwd", "max_win"}
78
79
80      def test_control_is_not_learned():
81          """The replay must use the pure-stdlib CONTROL variant (no numpy/LogisticRegression
? D5)."""
82          from backend.eval.experiment import CONTROL
83          assert CONTROL.is_learned() is False
84
85
86      def test_perturbing_trajectory_changes_segments(tmp_path):
87          """Mutating a copy of a trajectory and replaying yields DIFFERENT segments ? proving
the
88          windows are RE-DERIVED from the CSV each run, not merely echoed from
expected.json."""
89          slug = SLUGS[0]
90          src = gf.FIXTURES_DIR / slug
91          work = tmp_path / slug
92          work.mkdir()
93          # Copy the fixture, then TRUNCATE the trajectory to its first half so the derived
window's
94          # `end` shrinks ? a guaranteed structural change in the re-derived segments.
(Deleting an
95          # interior hole would not move start/end, since merge_gap coalesces across it and
the frame
96          # indices of the surviving rows are unchanged.)
97          lines = (src / "trajectory.csv").read_text(encoding="utf-8").splitlines()
98          header, body = lines[0], lines[1:]
99          mutated = [header] + body[: len(body) // 2]
100         (work / "trajectory.csv").write_text("\n".join(mutated) + "\n", encoding="utf-8",
newline="\n")
101         (work / "labels.csv").write_text((src / "labels.csv").read_text(encoding="utf-8"),
```

```
102                                         encoding="utf-8", newline="\n")
103         (work / "provenance.json").write_text((src /
"provenance.json").read_text(encoding="utf-8"),
104                                               encoding="utf-8", newline="\n")
105
106         original = gf.load_fixture(slug)["expected"]
107         perturbed = gf.replay_fixture(work)
108         assert perturbed["control_segments"] != original["control_segments"], (
109             "perturbed trajectory produced identical segments ? replay is echoing, not re-
deriving")
110
111
112     def test_freeze_is_idempotent(tmp_path):
113         """Re-freezing every built-in fixture into a temp dir produces expected.json
structurally
114         equal to the committed one (correct-by-construction + deterministic)."""
115         slugs = gf.freeze_synthetic(fixtures_dir=tmp_path)
116         assert slugs, "freeze_synthetic produced no fixtures"
117         for slug in slugs:
118             committed = gf.load_fixture(slug)["expected"]
119             refrozen = json.loads((tmp_path / slug /
"expected.json").read_text(encoding="utf-8"))
120             # Structural (parsed) equality ? no float churn, exact same shape/values.
121             assert gf.compare_expected(refrozen, committed) == [], (
122                 f"{slug}: re-frozen expected.json differs from committed (non-idempotent
freeze)")
123             assert refrozen == committed, f"{slug}: re-frozen expected.json not structurally
identical"
124
125
126     def test_check_cli_passes(capsys):
127         """The --check CLI replays all fixtures and exits 0 when they match the committed
snapshots."""
128         rc = gf.main(["--check"])
129         assert rc == 0
130
```

16. user (2026-06-16T06:27:24.229Z)

```
1       """Golden Regression Fixtures (P1 + M2) ? de-attribution / minimization + the refusal
gate.
2
3       These tests exercise ``golden_fixtures.freeze_real_fixture`` (the P1 tooling that turns
an OWNED,
4       minors-free real trajectory + golden labels into a committable, de-attributed Tier-0
fixture) and
5       its M2 ``--license`` / ``--minors-free`` / ``--owned`` refusal gate.
6
7       Everything runs on SYNTHETIC input written into ``tmp_path`` ? NO real data, NO network,
NO GPU.
8       We reuse M1's synthetic generators (``builtin_specs`` / ``make_synthetic_trajectory``
and the
9       ``write_trajectory_csv`` / ``write_labels_csv`` writers) to manufacture the "source"
CSVs, then
10      prove:
11        * the refusal gate blocks every unsafe combination (M2);
12        * a successful freeze is de-attributed (random ``fx_`` slug, no id/name/path, no
FORBIDDEN_KEYS);
13        * the time-origin reset is metric-INVARIANT (score identical with/without; only
absolute
14          ``control_segments`` shift toward 0);
15        * the freeze round-trips (``replay_fixture`` + ``compare_expected`` == []).
16
17      §6 honest-limits caveat: a green suite proves only that the deterministic post-
trajectory transforms
```

```
    are unchanged for the frozen cases ? characterization is behaviour-locking, NOT
correctness.
    """
    import pytest

    from backend.eval import golden_fixtures as gf


    # ---------------------------------------------------------------------------
    # Helpers ? manufacture a SYNTHETIC "real" source (trajectory + labels CSV) in tmp_path.
    # ---------------------------------------------------------------------------


    def _write_source(tmp_path, *, frame_offset: int = 0):
        """Write a synthetic source trajectory+labels CSV pair into ``tmp_path``.

        Reuses M1's first built-in scenario (a clean single rally), optionally shifted by
        ``frame_offset`` frames (and the matching label seconds) to model an arbitrary
absolute
        capture timeline. Returns ``(traj_path, labels_path, fps, frame_width, gts)``.
        """
        spec = gf.builtin_specs()[0]  # clean_single_rally
        fps, fw = spec.fps, spec.frame_width
        rows = gf.make_synthetic_trajectory(spec)
        if frame_offset:
            rows = [(f + frame_offset, v, x, y) for (f, v, x, y) in rows]
        gts = list(spec.gts)
        if frame_offset:
            off_sec = frame_offset / fps
            gts = [(s + off_sec, e + off_sec) for (s, e) in gts]

        traj = tmp_path / "src_trajectory.csv"
        labels = tmp_path / "src_labels.csv"
        gf.write_trajectory_csv(rows, traj)
        gf.write_labels_csv(gts, labels)
        return traj, labels, fps, fw, gts


    # ---------------------------------------------------------------------------
    # M2 ? refusal gate
    # ---------------------------------------------------------------------------


    @pytest.mark.parametrize(
        "kwargs, why",
        [
            ({"minors_free": False, "owned": True, "license": "owned"},
"minors_free=False"),
            ({"minors_free": True, "owned": False, "license": "owned"}, "owned=False"),
            ({"minors_free": True, "owned": True, "license": ""}, "license=''"),
            ({"minors_free": True, "owned": True, "license": "   "}, "license=blank"),
            ({"minors_free": True, "owned": True, "license": None}, "license=None"),
        ],
    )
    def test_refusal_gate_blocks_unsafe(tmp_path, kwargs, why):
        """freeze_real_fixture refuses (ValueError) unless minors_free AND owned AND a non-
blank license.

        The refusal is the code-enforced provenance gate (§4c / §7 ?): the unsafe public-
fixture path
        must be impossible to reach silently.
        """
        traj, labels, fps, fw, _ = _write_source(tmp_path)
        with pytest.raises(ValueError) as ei:
            gf.freeze_real_fixture(
```

```
                "scenario", traj, labels,
                fps=fps, frame_width=fw, fixtures_dir=tmp_path / "fixtures", **kwargs)
        # RefusalError is a ValueError subclass; the message must name a reason.
        assert isinstance(ei.value, gf.RefusalError), why
        assert "REFUSED" in str(ei.value), why
        # Nothing was written for a refused freeze.
        assert not (tmp_path / "fixtures").exists() or not any((tmp_path /
"fixtures").iterdir()), (
            f"{why}: a refused freeze must not write any fixture")


    # ---------------------------------------------------------------------------------------
    # P1 ? de-attribution
    # ---------------------------------------------------------------------------------------


    def test_real_fixture_is_deattributed(tmp_path):
        """A successful safe freeze is de-attributed: kind=cleared_real, fx_-prefixed
surrogate slug,
        no id/name/filename/source-path key, and NO FORBIDDEN_KEYS in expected.json."""
        fixtures = tmp_path / "fixtures"
        traj, labels, fps, fw, _ = _write_source(tmp_path)
        expected = gf.freeze_real_fixture(
            "clean_rally", traj, labels,
            license="owned", minors_free=True, owned=True,
            fps=fps, frame_width=fw, fixtures_dir=fixtures)

        slug = expected["slug"]
        assert slug.startswith("fx_"), f"surrogate slug must be opaque fx_<hex>, got
{slug!r}"

        prov = gf._read_json(fixtures / slug / "provenance.json")
        assert prov["kind"] == "cleared_real"
        assert prov["minors_free"] is True and prov["owned"] is True
        assert prov["license"] == "owned"
        assert prov["slug"] == slug
        # No attribution keys / no source filename anywhere in the provenance.
        for banned in ("video_id", "name", "filename", "path", "match", "capture_date"):
            assert banned not in prov, f"provenance must not carry an attribution key
{banned!r}"
        prov_text = (fixtures / slug /
"provenance.json").read_text(encoding="utf-8").lower()
        for tok in ("src_trajectory", "src_labels", str(traj).lower(), str(labels).lower()):
            assert tok not in prov_text, f"source token {tok!r} leaked into provenance"

        # expected.json carries no biometric/identity/non-shuttle stream.
        exp_text = (fixtures / slug / "expected.json").read_text(encoding="utf-8").lower()
        for forbidden in gf.FORBIDDEN_KEYS:
            assert forbidden not in exp_text, f"FORBIDDEN_KEYS leak {forbidden!r} in
expected.json"

        # The fixture is registered in the manifest with kind=cleared_real.
        manifest = gf.load_manifest(fixtures)
        rows = [e for e in manifest if e["slug"] == slug]
        assert len(rows) == 1 and rows[0]["kind"] == "cleared_real"


    def test_supplied_slug_is_honored(tmp_path):
        """An explicitly supplied slug is used verbatim (the surrogate map is the caller's
concern)."""
        fixtures = tmp_path / "fixtures"
        traj, labels, fps, fw, _ = _write_source(tmp_path)
        expected = gf.freeze_real_fixture(
            "scenario", traj, labels, slug="fx_custom00",
            license="owned", minors_free=True, owned=True,
```

```
            fps=fps, frame_width=fw, fixtures_dir=fixtures)
        assert expected["slug"] == "fx_custom00"
        assert (fixtures / "fx_custom00" / "expected.json").is_file()


    def test_freeze_real_does_not_clobber_existing_manifest(tmp_path):
        """Freezing a second real fixture must APPEND to the manifest, not wipe the
first."""
        fixtures = tmp_path / "fixtures"
        traj, labels, fps, fw, _ = _write_source(tmp_path)
        e1 = gf.freeze_real_fixture("a", traj, labels, slug="fx_aaaa0001",
                                    license="owned", minors_free=True, owned=True,
                                    fps=fps, frame_width=fw, fixtures_dir=fixtures)
        e2 = gf.freeze_real_fixture("b", traj, labels, slug="fx_bbbb0002",
                                    license="owned", minors_free=True, owned=True,
                                    fps=fps, frame_width=fw, fixtures_dir=fixtures)
        slugs = {e["slug"] for e in gf.load_manifest(fixtures)}
        assert {e1["slug"], e2["slug"]} <= slugs


    def test_positive_privacy_assertion_catches_leak_in_source_note(tmp_path):
        """source_note is non-identifying by contract ? a path/name in it trips the positive
scan."""
        fixtures = tmp_path / "fixtures"
        traj, labels, fps, fw, _ = _write_source(tmp_path)
        with pytest.raises(gf.RefusalError):
            gf.freeze_real_fixture(
                "scenario", traj, labels, slug="fx_leak0001",
                license="owned", minors_free=True, owned=True,
                fps=fps, frame_width=fw, fixtures_dir=fixtures,
                source_note=str(traj))  # leaking the source path ? must be refused


    # ---------------------------------------------------------------------------------------
    # P1 (d) ? time-origin reset is metric-invariant
    # ---------------------------------------------------------------------------------------


    def test_time_origin_reset_is_metric_invariant(tmp_path):
        """A large-offset source frozen WITH reset has IDENTICAL score to the SAME source
frozen
        WITHOUT reset ? but its absolute control_segments shift toward 0 (severed
timeline)."""
        offset_frames = 9000  # +5 minutes at 30fps ? a large absolute capture-timeline
offset
        traj, labels, fps, fw, _ = _write_source(tmp_path, frame_offset=offset_frames)

        with_reset = gf.freeze_real_fixture(
            "scenario", traj, labels, slug="fx_reset_on",
            license="owned", minors_free=True, owned=True,
            fps=fps, frame_width=fw, fixtures_dir=tmp_path / "with",
            time_origin_reset=True)
        without_reset = gf.freeze_real_fixture(
            "scenario", traj, labels, slug="fx_reset_off",
            license="owned", minors_free=True, owned=True,
            fps=fps, frame_width=fw, fixtures_dir=tmp_path / "without",
            time_origin_reset=False)

        # (1) Score is bit-identical ? metrics.score is built from pure differences.
        assert with_reset["score"] == without_reset["score"], (
            "time-origin reset must be metric-invariant (score built from pure
differences)")

        # (2) Absolute timeline differs ? the reset pulls control_segments toward 0.
        seg_on = with_reset["control_segments"]
```

```
195         seg_off = without_reset["control_segments"]
196         assert len(seg_on) == len(seg_off) and len(seg_on) >= 1
197         assert seg_on[0][0] < seg_off[0][0], "reset must shift the first segment start
toward 0"
198         # The shift equals the offset in seconds (within rounding).
199         shift = seg_off[0][0] - seg_on[0][0]
200         assert abs(shift - offset_frames / fps) < 1e-3, (
201             f"segment shift {shift} should equal the frame offset in seconds {offset_frames
/ fps}")
202         # Reset trajectory starts at frame 0.
203         first_frame = int((tmp_path / "with" / "fx_reset_on" / "trajectory.csv")
204                           .read_text(encoding="utf-8").splitlines()[1].split(",")[0])
205         assert first_frame == 0, "with reset, the trajectory must start at Frame 0"
206
207
208     # ---------------------------------------------------------------------------------
209     # P1 (e/f) ? round-trip / correct-by-construction
210     # ---------------------------------------------------------------------------------
211
212
213     def test_freeze_real_round_trips_check(tmp_path):
214         """After freeze_real_fixture, replay_fixture + compare_expected vs the written
expected.json
215         returns [] ? idempotent / correct-by-construction (the same path M1's --check
uses)."""
216         fixtures = tmp_path / "fixtures"
217         traj, labels, fps, fw, _ = _write_source(tmp_path)
218         expected = gf.freeze_real_fixture(
219             "scenario", traj, labels, slug="fx_rt000001",
220             license="owned", minors_free=True, owned=True,
221             fps=fps, frame_width=fw, fixtures_dir=fixtures)
222
223         fdir = fixtures / expected["slug"]
224         committed = gf._read_json(fdir / "expected.json")
225         recomputed = gf.replay_fixture(fdir)
226         assert gf.compare_expected(recomputed, committed) == [], (
227             "a just-frozen real fixture must replay-compare clean (correct-by-
construction)")
228
229
230     def test_freeze_real_uses_strict_label_loader(tmp_path):
231         """The labels are read through the STRICT gt_loader ? a malformed label row fails
loudly,
232         never silently shrinks the GT set."""
233         fixtures = tmp_path / "fixtures"
234         traj, labels, fps, fw, _ = _write_source(tmp_path)
235         # Corrupt one label row (end <= start) ? strict loader must raise.
236         labels.write_text("start,end\n5.0,1.0\n", encoding="utf-8", newline="\n")
237         with pytest.raises(ValueError):
238             gf.freeze_real_fixture(
239                 "scenario", traj, labels, slug="fx_strict01",
240                 license="owned", minors_free=True, owned=True,
241                 fps=fps, frame_width=fw, fixtures_dir=fixtures)
242
```

17. user (2026-06-16T06:27:24.559Z)

```
1       name: CI
2
3       on:
4         push:
5           branches: [master]
6         pull_request:
7
8       permissions:
```

```
9         contents: read
10
11      concurrency:
12        group: ci-${{ github.ref }}
13        cancel-in-progress: true
14
15      jobs:
16        # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17        # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
18        # tracked .py, including scripts/ and training/, which the backend-only
19        # py_compile smoke sweep does not reach.
20        lint:
21          name: Ruff lint
22          runs-on: ubuntu-latest
23          steps:
24            - uses: actions/checkout@v6
25            - uses: actions/setup-python@v6
26              with:
27                python-version: "3.13"
28            - name: Install ruff
29              run: python -m pip install ruff
30            - name: Ruff check
31              run: ruff check . --output-format github
32
33        # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
34        # per-module quarantine list ? ratchet by removing entries as modules clean
35        # up. Typed core deps are installed so mypy sees their real signatures
36        # (without them ignore_missing_imports would Any-stub half the checks).
37        typecheck:
38          name: Mypy (lenient baseline)
39          runs-on: ubuntu-latest
40          steps:
41            - uses: actions/checkout@v6
42            - uses: actions/setup-python@v6
43              with:
44                python-version: "3.13"
45            - name: Install mypy + typed core deps
46              run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
genai numpy
47            - name: Mypy
48              run: mypy backend training
49
50        # Fast guard against the 7fbbeb0 class of breakage: a SyntaxError or
51        # module-level import error shipping while the suite stays green.
52        # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
53        # on machines without the GPU stack (test_import_smoke stubs what's absent).
54        import-smoke:
55          name: Syntax sweep + import smoke (no GPU stack)
56          runs-on: ubuntu-latest
57          steps:
58            - uses: actions/checkout@v6
59            - uses: actions/setup-python@v6
60              with:
61                python-version: "3.13"
62            - name: Install minimal deps
63              run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
64            - name: Run import smoke tests
65              run: python -m pytest tests/test_import_smoke.py -v
66
67        full-suite:
68          name: Full test suite (offline, mocked)
69          runs-on: ubuntu-latest
70          steps:
71            - uses: actions/checkout@v6
```

```
72            - uses: actions/setup-python@v6
73              with:
74                python-version: "3.13"
75                cache: pip
76            - name: System libs for opencv-python
77              run: |
78                sudo apt-get update
79                sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y
libgl1 libglib2.0-0
80            - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
81              # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels
82              # need an out-of-band index that PEP 621 can't express. Keep installing
83              # them separately here, before the editable install pulls the rest.
84              run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
85            - name: Install package (editable) + dev tooling
86              # Editable PEP 621 install: runtime deps + the `dev` group
87              # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
88              run: python -m pip install -e .[dev]
89            # obsreport (private, soft dep) is absent here: reporting tests skip via
90            # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
91            # is therefore calibrated against THIS lane's number (local runs with
92            # obsreport installed measure a little higher).
93            # Floor calibrated 2026-06-11 against this lane's measured 72% (local
94            # with obsreport: 73%). Ratchet it UP as coverage grows; never down
95            # without an owner decision.
96            - name: Run full suite (with coverage floor)
97              run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
98
99        # Cross-OS determinism guard for the committed golden regression fixtures
100       # (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib
101       # (only numpy, no cv2/torch), so this is a FAST, focused job ? it does NOT run
102       # the full suite x3 OSes. It replays the Windows/Linux/macOS runner against the
103       # committed (parse-compared) snapshots, catching any cross-OS float drift beyond
104       # the 1e-6 tolerance before it can flake a contributor's PR on another machine.
105       golden-crossos:
106         name: Golden fixtures cross-OS (${{ matrix.os }})
107         strategy:
108           fail-fast: false
109           matrix:
110             os: [ubuntu-latest, windows-latest, macos-latest]
111         runs-on: ${{ matrix.os }}
112         steps:
113           - uses: actions/checkout@v6
114           - uses: actions/setup-python@v6
115             with:
116               python-version: "3.13"
117           - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
118             run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy
119           - name: Golden-fixtures characterization (cross-OS determinism guard)
120             run: |
121               python -m backend.eval.golden_fixtures --check
122               python -m pytest tests/test_golden_fixtures.py -q
123
```

18. assistant (2026-06-16T06:27:28.535Z)

Now I have the critical CI picture. Key confirmed observation: the golden-crossos job runs `--check` + `tests/test_golden_fixtures.py` but NOT `tests/test_golden_real_fixtures.py`. Let me verify the imports chain to confirm whether `import golden_fixtures` works on a clean clone WITHOUT numpy, and check the fixtures and docs.

19. user (2026-06-16T06:27:33.143Z)

```
=== backend/eval/calibration.py ===
  (no numpy/scipy/sklearn/cv2/torch at top)
=== backend/eval/classifier.py ===
18:import numpy as np
=== backend/eval/gemini_refine.py ===
  (no numpy/scipy/sklearn/cv2/torch at top)
=== backend/eval/training.py ===
  (no numpy/scipy/sklearn/cv2/torch at top)
=== backend/eval/eval_stats.py ===
  (no numpy/scipy/sklearn/cv2/torch at top)
=== backend/eval/segmentation_metrics.py ===
  (no numpy/scipy/sklearn/cv2/torch at top)
=== backend/eval/score_calibration.py ===
  (no numpy/scipy/sklearn/cv2/torch at top)
=== backend/eval/calibrate_wasb.py ===
  (no numpy/scipy/sklearn/cv2/torch at top)
=== backend/eval/annotations.py ===
  (no numpy/scipy/sklearn/cv2/torch at top)
=== backend/eval/harness.py ===
  (no numpy/scipy/sklearn/cv2/torch at top)
=== backend/eval/windowing.py ===
  (no numpy/scipy/sklearn/cv2/torch at top)
```

20. user (2026-06-16T06:27:33.165Z)

```
1       """A tiny, dependency-free logistic-regression classifier (numpy only).
2
3       The offline learned segmenter (ADR-004) must run locally with no external API and
4       no heavy ML stack ? the project already depends on numpy, and sklearn is not
5       installed. This implements just enough: standardized features, balanced-class
6       gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained
7       model is a small, human-readable, version-controllable file.
8
9       It deliberately stays simple: the rally-window features are few and roughly
10      linearly separable (high player motion + sustained activity => real rally), so a
11      well-regularized linear model is a sensible, inspectable baseline. Swap in a
12      richer model later behind the same fit/predict_proba/save/load interface.
13      """
14
15      import json
16      from typing import List, Optional
17
18      import numpy as np
19
20
21      class LogisticRegression:
22          def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
23              self.lr = lr
24              self.epochs = epochs
25              self.l2 = l2
26              self.w: Optional[np.ndarray] = None
27              self.b: float = 0.0
28              self.mean: Optional[np.ndarray] = None
29              self.std: Optional[np.ndarray] = None
30              self.feature_names: Optional[List[str]] = None
31
32          @staticmethod
33          def _sigmoid(z: np.ndarray) -> np.ndarray:
34              # Clip for numerical stability (avoid overflow in exp).
35              return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))
36
37          def _standardize(self, X: np.ndarray) -> np.ndarray:
38              return (X - self.mean) / self.std
39
40          def fit(self, X, y, feature_names: Optional[List[str]] = None,
```

21. assistant (2026-06-16T06:27:37.132Z)

Confirmed: `golden_fixtures.py` imports `from backend.eval.experiment import CONTROL, VideoCandidates, predict`, and `experiment.py` imports `from backend.eval.classifier import LogisticRegression` at module top (line 49), and `classifier.py` does `import numpy as np` at module top (line 18). So `import golden_fixtures` triggers a numpy import on a clean clone.

The CI golden-crossos job DOES install numpy (line 118), so the committed CI is green. But the doc claims and the "pure-stdlib, no numpy" framing need verification. Let me check the fixtures and the doc claims now.

22. user (2026-06-16T06:27:39.325Z)

```
total 28
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 .
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 ..
-rw-r--r-- 1 avidu 197609 4018 Jun 16 11:54 README.md
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 clean_single_rally
-rw-r--r-- 1 avidu 197609 1557 Jun 16 11:54 manifest.json
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 multi_rally_with_deadtime
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 occlusion_break
=== manifest ===
[
  {
    "fps": 30.0,
    "frame_width": 1280.0,
    "kind": "synthetic",
    "paths": {
      "dir": "clean_single_rally",
      "expected": "clean_single_rally/expected.json",
      "labels": "clean_single_rally/labels.csv",
      "provenance": "clean_single_rally/provenance.json",
      "trajectory": "clean_single_rally/trajectory.csv"
    },
    "schema_version": 1,
    "slug": "clean_single_rally",
    "windowing": {
      "max_win": 0.0,
      "mg": 2.0,
      "mwd": 2.0,
      "vt": 0.02
    }
  },
  {
    "fps": 30.0,
    "frame_width": 1280.0,
    "kind": "synthetic",
    "paths": {
      "dir": "multi_rally_with_deadtime",
      "expected": "multi_rally_with_deadtime/expected.json",
      "labels": "multi_rally_with_deadtime/labels.csv",
      "provenance": "multi_rally_with_deadtime/provenance.json",
      "trajectory": "multi_rally_with_deadtime/trajectory.csv"
    },
    "schema_version": 1,
    "slug": "multi_rally_with_deadtime",
    "windowing": {
      "max_win": 0.0,
      "mg": 2.0,
      "mwd": 2.0,
      "vt": 0.02
    }
  },
  {
    "fps": 30.0,
```

```
    "frame_width": 1280.0,
    "kind": "synthetic",
    "paths": {
      "dir": "occlusion_break",
      "expected": "occlusion_break/expected.json",
      "labels": "occlusion_break/labels.csv",
      "provenance": "occlusion_break/provenance.json",
      "trajectory": "occlusion_break/trajectory.csv"
    },
    "schema_version": 1,
    "slug": "occlusion_break",
    "windowing": {
      "max_win": 0.0,
      "mg": 2.0,
      "mwd": 2.0,
      "vt": 0.02
    }
  }
]
=== dirs ===
eval_baselines/fixtures/clean_single_rally/:
expected.json
labels.csv
provenance.json
trajectory.csv

eval_baselines/fixtures/multi_rally_with_deadtime/:
expected.json
labels.csv
provenance.json
trajectory.csv

eval_baselines/fixtures/occlusion_break/:
expected.json
labels.csv
provenance.json
trajectory.csv
```

23. user (2026-06-16T06:27:39.693Z)

```
1       """Load hand-annotated ground-truth rally timestamps from CSV.
2
3       Annotation file format (one file per video):
4
5           start,end
6           00:01:12,00:01:39
7           102.5,131.0
8
9       - A header row `start,end` is optional (auto-detected and skipped).
10      - Each timestamp is either decimal seconds (`102.5`) or colon-separated
11        `MM:SS` / `HH:MM:SS` (`00:01:12`). Both forms may be mixed in one file.
12      - Blank lines and `#` comment lines are ignored.
13      """
14
15      import csv
16      import logging
17      from typing import List, Tuple
18
19      logger = logging.getLogger(__name__)
20
21      Interval = Tuple[float, float]
22
23
24      def parse_timestamp(value: str) -> float:
25          """Parse a timestamp string to float seconds.
```

```
26
27          Accepts decimal seconds or colon-separated MM:SS / HH:MM:SS.
28          Raises ValueError on anything unparseable so bad annotations fail loudly.
29          """
30          value = value.strip()
31          if not value:
32              raise ValueError("empty timestamp")
33          if ":" in value:
34              parts = value.split(":")
35              if len(parts) > 3:
36                  raise ValueError(f"too many ':' segments in timestamp '{value}'")
37              total = 0.0
38              for part in parts:                # most-significant first (H, M, S)
39                  total = total * 60 + float(part)
40              return total
41          return float(value)
42
43
44      def load_annotations(csv_path: str) -> List[Interval]:
45          """Load and validate ground-truth rally intervals from a CSV file.
46
47          Delegates to the canonical loader (gt_loader.py, ADR-008 prereq), which accepts BOTH
48          golden conventions: positional/headered ``start,end`` (the collector export) and the
49          rally-annotator ``rally_number,start_time,end_time,...`` schema. Strict: raises
50          ValueError on malformed rows (bad timestamp, start >= end) so labeling mistakes
51          surface immediately rather than silently skewing scores. Returns intervals sorted by
52          start time.
53          """
54          from backend.eval.gt_loader import load_gt_intervals
55          intervals = load_gt_intervals(csv_path, strict=True)
56          logger.info("Loaded %d ground-truth rallies from %s", len(intervals), csv_path)
57          return intervals
58
59
60      def write_scaffold(csv_path: str) -> None:
```

24. assistant (2026-06-16T06:27:43.181Z)

Now let me read the expected.json files and the doc claims (§4c/§5/§6), plus the README. Let me also examine the `net_crossings` float round-trip suspicion and the `parse_timestamp` round-trip for the time-origin reset.

25. user (2026-06-16T06:27:46.427Z)

```
=== clean_single_rally/expected.json ===
{
  "candidates": [
    {
      "end": 3.966667,
      "shuttle": {
        "active_ratio": 1.008475,
        "duration": 3.933333,
        "mean_velocity": 1.27678,
        "net_crossings": 7,
        "peak_velocity": 1.970041
      },
      "start": 0.033333
    }
  ],
  "control_segments": [
    [
      0.033333,
      3.966667
    ]
  ],
```

```
  "gt_hash": "6c93801fd607",
  "iou": 0.5,
  "schema_version": 1,
  "score": {
    "f1": 1.0,
    "false_negatives": 0,
    "false_positives": 0,
    "iou_threshold": 0.5,
    "mean_end_error": 0.033333,
    "mean_iou": 0.983333,
    "mean_start_error": 0.033333,
    "num_gt": 1,
    "num_pred": 1,
    "precision": 1.0,
    "recall": 1.0,
    "true_positives": 1
  },
  "slug": "clean_single_rally"
}

=== clean_single_rally/labels.csv ===
start,end
0.000000,4.000000
=== clean_single_rally/provenance.json ===
{
  "description": "One continuous ~4s rally; shuttle oscillates across the net ? 1 window.",
  "fps": 30.0,
  "frame_width": 1280.0,
  "iou": 0.5,
  "kind": "synthetic",
  "license": "synthetic",
  "minors_free": true,
  "note": "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only. Plumbing-
correctness fixture ? NOT a measure of rally-detection quality (§6).",
  "paths": {
    "expected": "expected.json",
    "labels": "labels.csv",
    "trajectory": "trajectory.csv"
  },
  "schema_version": 1,
  "slug": "clean_single_rally",
  "windowing": {
    "max_win": 0.0,
    "mg": 2.0,
    "mwd": 2.0,
    "vt": 0.02
  }
}
=== multi_rally_with_deadtime/expected.json ===
{
  "candidates": [
    {
      "end": 4.0,
      "shuttle": {
        "active_ratio": 1.008403,
        "duration": 3.966667,
        "mean_velocity": 1.282558,
        "net_crossings": 7,
        "peak_velocity": 1.970041
      },
      "start": 0.033333
    },
    {
      "end": 10.966667,
      "shuttle": {
```

```
        "active_ratio": 1.008475,
        "duration": 3.933333,
        "mean_velocity": 1.27678,
        "net_crossings": 7,
        "peak_velocity": 1.970041
      },
      "start": 7.033333
    }
  ],
  "control_segments": [
    [
      0.033333,
      4.0
    ],
    [
      7.033333,
      10.966667
    ]
  ],
  "gt_hash": "bbec003aed74",
  "iou": 0.5,
  "schema_version": 1,
  "score": {
    "f1": 1.0,
    "false_negatives": 0,
    "false_positives": 0,
    "iou_threshold": 0.5,
    "mean_end_error": 0.016667,
    "mean_iou": 0.9875,
    "mean_start_error": 0.033333,
    "num_gt": 2,
    "num_pred": 2,
    "precision": 1.0,
    "recall": 1.0,
    "true_positives": 2
  },
  "slug": "multi_rally_with_deadtime"
}

=== multi_rally_with_deadtime/labels.csv ===
start,end
0.000000,4.000000
7.000000,11.000000
=== multi_rally_with_deadtime/provenance.json ===
{
  "description": "Two ~4s rallies separated by 3s of still (resting-shuttle) dead-time ? 2
windows.",
  "fps": 30.0,
  "frame_width": 1280.0,
  "iou": 0.5,
  "kind": "synthetic",
  "license": "synthetic",
  "minors_free": true,
  "note": "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only. Plumbing-
correctness fixture ? NOT a measure of rally-detection quality (§6).",
  "paths": {
    "expected": "expected.json",
    "labels": "labels.csv",
    "trajectory": "trajectory.csv"
  },
  "schema_version": 1,
  "slug": "multi_rally_with_deadtime",
  "windowing": {
    "max_win": 0.0,
    "mg": 2.0,
```

```
    "mwd": 2.0,
    "vt": 0.02
  }
}
=== occlusion_break/expected.json ===
{
  "candidates": [
    {
      "end": 3.966667,
      "shuttle": {
        "active_ratio": 1.008475,
        "duration": 3.933333,
        "mean_velocity": 1.27678,
        "net_crossings": 7,
        "peak_velocity": 1.970041
      },
      "start": 0.033333
    },
    {
      "end": 10.966667,
      "shuttle": {
        "active_ratio": 1.008475,
        "duration": 3.933333,
        "mean_velocity": 1.27678,
        "net_crossings": 7,
        "peak_velocity": 1.970041
      },
      "start": 7.033333
    }
  ],
  "control_segments": [
    [
      0.033333,
      3.966667
    ],
    [
      7.033333,
      10.966667
    ]
  ],
  "gt_hash": "bbec003aed74",
  "iou": 0.5,
  "schema_version": 1,
  "score": {
    "f1": 1.0,
    "false_negatives": 0,
    "false_positives": 0,
    "iou_threshold": 0.5,
    "mean_end_error": 0.033333,
    "mean_iou": 0.983333,
    "mean_start_error": 0.033333,
    "num_gt": 2,
```

26. assistant (2026-06-16T06:27:51.199Z)

I notice `active_ratio` is 1.008475 (>1.0) ? a slightly odd value worth noting but it's deterministic. Now let me read the doc claims and check the critical determinism suspicions. Let me focus on the `net_crossings` float round-trip and the `parse_timestamp`/`_fmt` round-trip in the real-fixture reset path.

27. user (2026-06-16T06:27:52.527Z)

```
1       # Golden Regression Fixtures ? video-free, non-attributable regression suite
2
3       > **Status:** `IN PROGRESS` ? owner-gated · **Owner:** Avi · **Created:** 2026-06-16 ·
```

Last updated: 2026-06-16 · **Legal:** owner **accepted the researched risk** 2026-06-16 (conditions §3 stand; this is the owner's informed acceptance, **not** a retained-counsel opinion)

> **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE, per archive policy)

> **Tracking anchors:** §7 progress tracker is the source of truth; indexed in `docs/README.md`; pointer in memory `current-state` (and `docs/NEXT_STEPS.md` once work starts).

> **Relation to existing docs:** EXTENDS (does not supersede) `docs/GOLDEN_DATA_SHARING.md` (#175); reuses the eval stack documented in `docs/EVALUATION.md` / `docs/EXPERIMENT_HARNESS.md`; quality floor ties to `docs/QUALITY_ITERATIONS.md`.

> **Honesty note:** §3 (Legal) is **researched general information, NOT legal advice** ? it gates on retained counsel (§8). §2/§4/§7 mark which claims are **[verified]** against code vs **[design]** proposals.

---

0. TL;DR

Let a contributor on **another machine with no video, GPU, WSL, or DB** run the rally-segmentation regression suite from **committed structured files**, so refactors and non-quality PRs get a red/green side-effect signal. Two layers, built once and grown by accretion:

1. **Mechanics** ? freeze the deterministic pipeline at the **post-detector trajectory seam** and replay it through the *already-shipped* eval stack (`experiment.py` / `metrics.py` / `windowing.py` / `regression.py`), with two auto-discovered gates: a **characterization/snapshot** gate (catches any behaviour change) and a **quality-floor** gate (catches quality drops).
2. **Privacy/legal filter (SEALED-FIXTURES)** ? only **de-attributed, minimized, owner-source, biometric-free** numbers are ever committed; richer/sensitive streams stay key-gated or out-of-repo. Non-attributability comes from **deletion-at-source**, not encryption.

The convergence that makes this clean: the *privacy* answer (publish shuttle-only, drop person/pose streams) and the *determinism* answer (freeze only the pure-stdlib `CONTROL` path, keep the numpy/BLAS learned scorer off the cross-machine gate) point at the **same minimal public tier**. That coincidence is the spine of the design.

Two findings already actionable, independent of the rest:
- ? **Live de-attribution bug** *(verified)*: real player/venue names are committed today in `eval_baselines/heuristic_lovo_n6.json`, `eval_baselines/gemini_wrapper_2026-06-10.json`, and `backend/eval/eval_partial_labels.py` (the last also has a personal `C:/Users/avidu/...` path) ? and in git history. ? **Deliverable P0.**
- ? **Determinism trap** *(verified, corroborated by two independent red-teams)*: the learned variants (`shuttle_only`, `fusion`) train a numpy/OpenBLAS `DYNAMIC_ARCH` logistic regression whose decisions flip near the 0.5 threshold across CPUs ? would false-RED on a contributor's machine. ? the cross-machine gate must be **CONTROL/static-only**.

---

1. Problem & goal

The rally-quality measurement is split across two machines by capability (`docs/GOLDEN_DATA_SHARING.md`):

- `backend/eval/fusion_golden.py` **extracts** `*_golden_features.json` ? needs the **video + trajectory CSV + labels + GPU/WSL** (GPU box only).
- `backend/eval/fusion_compare.py` / `backend/eval/ablation.py` **re-score** those JSONs on **pure CPU, no video** ? any machine.

The CPU re-scoring is *already* video-free, but the inputs live in gitignored `output/` / private OneDrive, so the regression tests that depend on them **skip everywhere** (`tests/test_ablation.py::test_litmus_reproduces_gate_a`, `tests/test_fusion_compare.py`). The committed `eval_baselines/` baselines exist but `regression.py` still scores DB-stored predictions (needs a pipeline run on the video).

Goal: commit a tiny, version-tagged, **numeric-only, non-attributable** per-fixture corpus + auto-discovered gates so a clean `git clone` / CI gets a real red/green regression

signal with **zero video/GPU/DB**, while never leaking attributable or sensitive data.
35
36   ---
37
38   ## 2. Decisions locked
39
40 | # | Decision | Source | Implication |
41 |---|----------|--------|-------------|
42 | D1 | **Corpus is owner-recorded / owned** | owner answer 2026-06-16 | Legal analysis takes **Fork A** (clean): no third-party copyright/ToS infringement; concern is trade-secret/moat + privacy. Strongest non-attributability (no public reference to correlate against). |
43 | D2 | **Repo private now, may open-source later** | owner answer 2026-06-16 | Design for **public-grade hygiene from day one** ? git history is permanent; nothing scrubable-later. |
44 | D3 | **Adults, consent not formalized** | owner answer 2026-06-16 | **Person/pose/gait streams stay OUT of the public tier** (DPDP/GDPR derived-personal-data risk without consent). Public tier = **shuttle + abstract intervals + non-biometric features only**. |
45 | D4 | **Freeze seam = post-detector trajectory** *(design)* | workflow-1 synthesis | Widest deterministic surface; reuse the shipped eval stack verbatim; the raw per-frame CSV is **never committed** (only ~5 aggregate, fps/res-invariant shuttle scalars/window). |
46 | D5 | **Cross-machine gate is CONTROL/static-only** *(design)* | both red-teams [verified cause] | Learned variants (numpy/BLAS) are **owner-box/Tier-1 only**, asserted with tolerance, never as a cross-machine byte-snapshot. |
47 | D6 | **Non-attributability = deletion-at-source, encryption = public-only barrier** *(design)* | workflow-2 synthesis | A contributor who runs the tests necessarily reads plaintext+key (DRM/analog-hole). So we **delete** attribution, not hide it. Encryption only stops the no-key public. |
48 | D7 | **Two tiers** *(design)* | workflow-2 synthesis | **Tier-0** public/no-key (shuttle-only, de-attributed). **Tier-1** key-gated/out-of-repo (person/audio, full corpus), **skips cleanly when absent**. |
49
50   ---
51
52   ## 3. Legal foundation (researched ? gates on counsel, §8)
53
54   **Core verdict (settled across IN/US/EU):** the derived numbers ? per-frame shuttle (x,y), rally start/end intervals, audio/court/aggregate-person feature values ? **do NOT inherit the source video's copyright.** They are uncopyrightable *facts/measurements*, not authored expression, and not a §101 "derivative work."
55
56 | Jurisdiction | Copyright of the data | Database right | Contract / ToS | Net |
57 |---|---|---|---|---|
58 | **India** (primary) | Not copyrightable as facts (*EBC v. D.B. Modak*); but compilations get thin protection on a low "skill/labour" bar (*Burlington*, *OLX v. Padawan*) ? only bites if **lifted from a 3rd-party dataset**, not self-derived | **None** (no sui-generis right) ? a real permissiveness edge | Anti-scraping ToS (untested in court) + **IT Act s.43/s.66** civil/criminal exposure for 3rd-party ingestion | **Owned-source + minors-excluded + no person/pose = clean** |
59 | **US** | Not copyrightable (*Feist*; §102(b); *NBA v. Motorola*) | None (*Feist* killed sweat-of-the-brow) | **Dominant risk.** ToS enforceable where copyright/CFAA fail (*ProCD*, *hiQ* ? injunction + **forced deletion**). Remedy for redistribution is takedown, not nominal damages | Publishable as facts **iff** owned/permissioned source + no barring ToS + no person/biometric/minor stream |
60 | **EU / UK** | Not copyrightable (*Football Dataco v. Yahoo*) | **Genuinely contestable.** *Football Dataco v. Sportradar*: recording pre-existing on-court facts = "**obtaining**", not "creating" ? our "we created it" premise is **overconfident**; harm test = *CV-Online Latvia* | Ryanair v. PR Aviation: ToS can ban extraction/redistribution even with no IP right | Get EU+UK advice before redistributing numeric streams at scale |
61
62   **Provenance is the decisive fork.** **Fork A (owner-recorded, unpublished)** ? the only path safe to commit publicly; collapses to a trade-secret *choice* (Indian breach-of-confidence / Contract Act 1872 / IT Act s.72 ? **not** US DTSA) + privacy. **Fork B (broadcast/YouTube/scraped)** ? do **not** commit; the extraction act needs a license or now-**contested** US fair use (*Thomson Reuters v. Ross* 2025; USCO May-2025), and unlawful acquisition independently sinks it (*Bartz v. Anthropic*, ~$1.5B settlement). Caveat: owner

footage **shot at a third party's event** (tournament/club) carries ticket/venue/organiser/broadcaster terms ? treat as Fork B until cleared.

Publish-by-stream (privacy):

- ? **Publish-OK:** shuttle (x,y)+Visibility (inanimate object), abstract rally/GT **intervals** (identity-stripped), non-biometric audio/court/aggregate-person features detached from identity.
- ?? **Conditional (effectively never in v1):** person bounding boxes that *persistently single out a player* ? flips to GDPR Art.9 the moment it identifies. This architecture is prone to that flip ? treat per-player tracking as a tripwire.
- ? **Never publish:** pose/skeleton/**gait** (re-identifies at 80?95% even with faces blurred ? *Weiss et al. 2025*), face crops, any named-player?movement mapping. Matches the existing "gait/biometrics strictly future (GDPR Art.9)" guardrail.
- ? **Minors flip everything:** DPDP "child" = under 18; s.9(3) **prohibits behavioural monitoring** (the Rule-12 exemptions don't cover this project). A persistent per-player signature on a minor is, conservatively, prohibited. **Exclude minors** from any persisted stream beyond inanimate shuttle/interval data.

Non-attributability is relative, not absolute. A per-video trajectory is a fingerprint (de Montjoye: 4 points ? 95% re-ID). It's only non-attributable because the **Fork-A source stays unpublished** (no public reference to correlate against). Per *EDPS v. SRB* (CJEU C-413/23 P, 2025) it **remains personal data to the holder** who can re-run the detector. So: conditional, revocable (publish the source and the link snaps back), forward-looking (re-assess as the public-video environment grows). Never market it as "anonymous."

Key authorities: Feist 499 U.S. 340; NBA v. Motorola 105 F.3d 841; 17 U.S.C. §101/§102(b); Sega v. Accolade 977 F.2d 1510; Football Dataco v. Yahoo (C-604/10) & v. Sportradar (C-173/11); BHB v. William Hill (C-203/02); CV-Online Latvia (C-762/19); Ryanair v. PR Aviation (C-30/14); Thomson Reuters v. Ross (D. Del. 2025); EBC v. D.B. Modak; DPDP Act 2023 s.3/s.9 + Rules 2025; EDPS v. SRB (C-413/23 P). Full URLs in the research artifact (§10).

---

4. Architecture (design)

4a. Freeze seam + reuse map

Freeze at the **trajectory CSV ? pure-Python replay**. One shared helper `backend/eval/golden_fixtures.py::replay_fixture()` is the *single* code path used by both the freeze tool and both gates:

`parse_trajectory_csv ? windowing.build_windows(preset) ? distill_local.build_candidates ? rally_gate net_crossings ? experiment.predict(CONTROL) ? merge_overlaps ? metrics.score`.

Reuses **verbatim**: `metrics.score/interval_iou`, `segmentation_metrics`, `experiment.predict/compare/Variant.spec_hash`, `windowing.WindowingPreset` (default SERVED), `regression.gt_hash/save_baseline + incommensurability guards`, `eval_stats` (seed=0), `eval_baselines/` committed-baseline precedent.

4b. Tiered model (SEALED-FIXTURES)

Tier	Where	Key?	Contents	Test behaviour
Tier-0	`eval_baselines/fixtures/` (tracked)	none	opaque `fixture_id`, fps/frame_width (quantized), relative `start`/`end`, **5 shuttle scalars**, label, relative `gts`. **No person/audio/pose.**	**Unconditional** ? runs on every PR/fork; turns today's skipped litmus into a real public gate
Tier-1	OneDrive (default) **or** SOPS-encrypted in-repo	token/age key	full corpus **with** person/audio; authoritative ?F1/CI/sign-test + exact Gate-A litmus	**Skips cleanly** when key/data absent (generalize `_golden_present()` ? `_tier1_present()`)

A fresh no-key clone is always **green**; coverage never depends on a secret.

4c. Anti-attribution measures (Tier-0)

- **Opaque RANDOM surrogate id** (`os.urandom` + a private surrogate?source map kept off-git) ? *not* the MD5 `video_id`, and *not* HMAC(salt, video_id?name) (red-team: brute-forceable over a tiny known roster if the salt leaks). Strip the `name` field (today = source filename).
- **Metadata strip** ? no filename, video_id, match/event/player identity, capture date.

- **Time-origin reset** ? subtract per-fixture `t0` from all start/end/gts. **Provably metric-invariant** (`metrics.interval_iou` and start/end-error are pure differences) ? bit-identical scores, severed absolute timeline.
- **Stream minimization** ? Tier-0 keeps shuttle block only; person/audio excluded; pose/gait/face have **no field in the schema by design**.
- **Raw-trajectory exclusion** ? never commit the per-frame `(Frame,Visibility,X,Y)` CSV.
- **Quantize fps/frame_width** ? red-team: at n=6 the raw values are a camera/venue quasi-identifier.
- **Provenance hard-fail gate** ? the de-attribution pass refuses to emit a public fixture for any record not tagged owner-recorded(Fork-A) + minors-excluded + biometric-excluded.

4d. The two gates

- **Characterization/snapshot** ? replay fixture, **parse-compare** JSON (never byte-compare ? CRLF/float-format safe; `core.autocrlf=true` here), exact on int/structural fields, `approx(abs=1e-6)` on floats; stamp + check windowing-preset/label/gt fingerprints (incommensurable ? loud fail). Catches any behaviour change even at constant F1. **CONTROL path only** across machines.
- **Quality-floor** ? score vs committed labels; assert per-video + mean F1 ? floor in `eval_baselines/regression_baseline.json` (created here; carries a **windowing fingerprint** so a served-default change is flagged incommensurable, not silently compared); emit `eval_stats` paired-delta-CI + sign test (NUMBERS INFORM); add a **metrics-vs-base-branch** paired delta. Synthetic-tier floors measure **correctness, not field quality** (documented).

4e. Serialization & optional protobuf

JSON with `sort_keys=True` + pre-quantized floats + LF is sufficient. **Protobuf is optional (O1)** ? adopt only if cross-OS byte-stable snapshots are wanted (proto3 `rally.fixture.v1`, codegen + `protoc && git diff --exit-code` guard, `*.pb binary -text`). Protobuf is **encoding, not secrecy** (trivially `--decode_raw`).

4f. Encryption & keys (Tier-1 only)

Default = out-of-repo + token (existing OneDrive `RALLY_GOLDEN_DIR` seam ? zero sensitive bytes in git history, sidesteps the legal "redistribution" act). **Alternative = SOPS + age** (per-recipient, revocable, value-level diffs) for Fork-A-clean-but-private data wanting in-repo versioning. Rejected: git-crypt (whole-repo re-key), git-lfs (storage ? secrecy). CI secret exposed **only to the Tier-1 job on trusted branches, never to fork-PR runners**. The **HMAC/surrogate salt** (if used) and the surrogate?source map live with the private corpus, rotated per release. Document explicitly that Tier-1 crypto defends **only** the no-key public.

---

5. Threat model (who's protected vs not)

Actor	Protected against	NOT protected (by design / residual)
TM1 no-key public	map fixture?source/match/player; decrypt Tier-1; confirm a source even holding the candidate file	read Tier-0 numbers + run shuttle-only harness (intended; useless-in-isolation)
TM2 keyed contributor	recover filename/video_id/match/face/pose/gait ? *never written* (survives the analog hole)	read every value the suite reads (unpreventable; not pretended otherwise)
TM3 CI runner	leaked log can't leak attribution it never had	runs the suite (intended); Tier-1 secret never on fork runners
TM4 malicious keyed insider	correlation needs a public reference; Fork-A source is unpublished ? "not reasonably likely"	**residual:** conditional + revocable ? publish/leak the source later and the link can snap back (forward-looking re-test duty)
TM5 re-ID adversary	source unpublished + only ~5 invariant scalars/window + reset timeline ? no back-match	becomes live if a Fork-A source is ever published
TM6 accidental plaintext commit	required CI leak-scan rejects MD5/known-name/non-surrogate/plaintext-SOPS	if the required check is disabled, a leak lands

---

6. Honest limits ? what a green suite does NOT prove

127   - Green proves only that the **deterministic post-trajectory transforms** are unchanged for the **frozen cases**. It does **not** cover: the WASB/TrackNet detector, `_probe_fps_width`/cv2 frame-seek, the AI/provider handoff, chunking/re-encode, the ffmpeg stitch, DB persistence/migrations, or the API filter. *A refactor that breaks decoding or the detector passes these gates green.* The owner-box `regression.py` DB run remains the only full-chain check.
128   - Characterization = **behaviour-locking, not correctness** ? it freezes current behaviour *including current bugs*. "Passing" = "behaviour unchanged", never "behaviour correct". This caveat must ride in the **test pass/fail message**, not only the docs.
129   - **Synthetic-tier floors = plumbing correctness, NOT field quality.** Never cite synthetic F1 in `QUALITY_ITERATIONS.md` / the paper as rally-detection performance (tag `kind: synthetic`).
130   - Tier-0 public coverage is **shuttle-only**; the person/audio fusion ?F1 and exact Gate-A values need Tier-1 (skipped on no-key clones).
131   - Data is **not "anonymous"** absolutely (EDPS v. SRB) ? relative, conditional, revocable, forward-looking.
132   - Encryption gives **no secrecy from a running contributor**; protobuf gives **no secrecy at all**.
133   - De-attribution does **not** cure provenance/licensing ? it sits *behind* the per-record provenance gate + counsel sign-off.
134   - `--refresh-snapshot` can rubber-stamp a real regression; mitigation is **review of the visible diff** + a separate, deliberate floor-baseline update. Require a recorded reason.
135
136   ---
137
138   ## 7. Deliverables & progress tracker ? **source of truth**
139
140   Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. One **small PR per row**, branched from `origin/master`, no stacking. Update the row (status + PR link) as work lands.
141
142 | ID | Deliverable | Depends on | Counsel-gated? | Status | PR |
143 |----|-------------|-----------|----------------|--------|----|
144 | **P0** | **Leak remediation + required CI leak-scan.** Scrub real names/paths in `eval_baselines/heuristic_lovo_n6.json`, `gemini_wrapper_2026-06-10.json`, `backend/eval/eval_partial_labels.py`, `tests/test_cleanup_caches.py` ? opaque surrogate keys + private map; add CI leak-scan (MD5 / known-name / non-surrogate / plaintext-SOPS) as a **required status check**; decide on git history (PR #77, accept vs `filter-repo`). | ? | No (owned data hygiene) | ? | ? |
145 | **M1** | **Fixture framework + synthetic Tier-0 + shared replay helper** (also delivered M2-core + M3 ? see below). `backend/eval/golden_fixtures.py` (`replay_fixture`, schema dispatch, `Path(__file__)`-relative) + 3 **tool-generated** synthetic fixtures + manifest + README + `.gitattributes` LF pin + `tests/test_golden_fixtures.py`. CONTROL-only / pure-stdlib; cross-OS verified. | ? | No (synthetic) | ? | [#189](https://github.com/avidullu/badminton-highlight-indexer/pull/189) |
146 | **M2** | **Freeze/refresh extractor.** Core (`freeze_synthetic` / `refresh_snapshot` / `--check` CLI + idempotence test) shipped in M1 (#189); the `--license`/`--minors-free`/`--owned` **refusal gate** shipped with P1 (this PR). | M1 | No | ? | [#189](https://github.com/avidullu/badminton-highlight-indexer/pull/189) · _this PR_ |
147 | **M3** | **Characterization gate** ? `tests/test_golden_fixtures.py`: parse-compared (exact-int / approx-float 1e-6), **CONTROL-only**, positive no-person/audio assertion, perturbation + idempotence. **Shipped in M1 (#189).** (Windowing/label fingerprint guard + row-count cap ? fold into M4.) | M1 | No | ? | [#189](https://github.com/avidullu/badminton-highlight-indexer/pull/189) |
148 | **MX** | **Cross-OS determinism guard (CI).** Lightweight `golden-crossos` CI job (ubuntu/windows/macos) running `--check` + the fixtures test ? locks the cross-OS guarantee (CONTROL replay is numpy-only, no GPU stack; a *separate fast job*, not full-suite×3). | M1 | No | ? | [#191](https://github.com/avidullu/badminton-highlight-indexer/pull/191) |
149 | **M4** | **Quality-floor gate + first `regression_baseline.json`.** `tests/test_golden_quality_floor.py`; create `eval_baselines/regression_baseline.json` with **windowing fingerprint**; metrics-vs-base paired-delta; corpus-level (not per-slug) LOVO for runtime. | M1, M2 | No | ? | ? |
150 | **P1** | **De-attribution + minimization** ? `golden_fixtures.freeze_real_fixture()` (extends the M1 module, not a separate script): **opaque random `fx_<hex>` surrogate slug**, metadata strip, metric-invariant **time-origin reset**, strict GT loader, **positive no-person/audio/MD5/source-path assertion**, manifest non-clobber; `--freeze-real` CLI with the

refusal gate (exit 2 unless owned + minors-free + license). Tooling-only, 12 synthetic tests; **no real data committed**. | M1 | No (tooling) | ? | _this PR_ |
151 | **P2** | **Owner-accepted (researched-risk), 2026-06-16.** Owner accepted the researched IP/privacy analysis (§3) to proceed **under its conditions** (owned/Fork-A source · minors excluded · biometric/person streams excluded · de-attributed). Owner's informed risk acceptance ? **NOT a retained-counsel opinion**; optional future counsel confirmation per §8. | P1 | Owner-accepted | ? | ? |
152 | **P3** | **Tier-0 REAL subset.** Commit de-attributed, minors/biometric-excluded, owned Fork-A real fixtures (unblocked by P2). Hard-gated **in code** by P1's refusal gate (owned + minors-free + license required) ? needs the owner's per-video clearance list. | P1, P2 | conditions §3 | ? | ? |
153 | **O1** | *(optional)* **Protobuf + deterministic serialization.** `rally.fixture.v1` proto3, codegen + `protoc`/`git diff --exit-code` guard, canonical writer, `.gitattributes` binary. Only if cross-OS byte-stable snapshots are wanted. | M1 | No | ? | ? |
154 | **O2** | *(optional)* **Tier-1 key-gated path.** `scripts/fetch_golden.py` (OneDrive token, default) and/or SOPS+age; `_tier1_present()`; `RALLY_GOLDEN_DIR` wired; Tier-1 CI job gated on secret, never on fork runners. | M1 | No | ? | ? |
155 | **DOC** | **Finalize this doc + cross-links.** Add `eval_baselines/fixtures/` row to `CODE_MAP §0` + TEST MAP; cross-link `EVALUATION.md` / `EXPERIMENT_HARNESS.md` / `GOLDEN_DATA_SHARING.md` / `QUALITY_ITERATIONS.md`; update `docs/README.md` index + memory `current-state`/handoff. | M1..M4 | No | ? | ? |
156
157   **Must-fix-before-any-public-commit (red-team), folded into the rows above:** ? numeric-tolerance not byte-equality + multi-OS matrix + CONTROL-only (M3/D5/MX ?); ? scrub names in **all** files + history decision (P0); ? leak-scan a **required** check, hooks are convenience only (P0); ? person-optional loader + **positive** no-person assertion, don't trust the silent `0.0` default (P1 ?); ? random surrogate id not HMAC-over-roster (P1 ?); ? quantize/drop fps/frame_width (P1 ? fps/fw retained as scale constants; quantize is a P3 option); ? document Tier-0 omits the fusion path (DOC); ? counsel + Fork-A + minors/biometric exclusion as hard preconditions (P2 owner-accepted; enforced in code by P1's refusal gate ?).
158
159   ### Progress log (quantitative ? coverage must hold/improve, CI incl. the 3-OS cross-OS job green)
160 | PR | Deliverable(s) | Suite | Coverage | Notes |
161 |----|----------------|-------|----------|-------|
162 | [#189](https://github.com/avidullu/badminton-highlight-indexer/pull/189) | M1 + M2-core + M3 | 746 pass / 7 skip | 80.41% | synthetic Tier-0 + CONTROL replay + characterization; cross-OS proven (CI Linux replayed Windows-frozen fixtures) |
163 | [#191](https://github.com/avidullu/badminton-highlight-indexer/pull/191) | MX + tracker/legal reconcile | full suite green | 80.41% | `golden-crossos` CI job green on ubuntu+windows+macos ? cross-OS **enforced**; legal ? owner-accepted |
164 | _this PR_ | P1 + M2 refusal gate | 759 pass / 7 skip | **80.49%** (+0.08) | `freeze_real_fixture` + refusal gate; **+13 tests**; tooling-only, no real data committed |
165
166   ---
167
168   ## 8. Open owner / counsel questions
169
170   **Counsel questions ? documented basis (the owner accepted the researched risk on 2026-06-16; see P2). These remain the record, and a retained-counsel confirmation is optional/recommended before scaling beyond a small owned, minors-free, de-attributed subset:**
171   1. Does the EU/UK **sui-generis database right** subsist in our self-derived shuttle/rally streams on the *Sportradar* "obtained vs created" reasoning, and does redistributing a de-attributed Tier-0 subset cause *CV-Online* harm?
172   2. Post *Thomson Reuters v. Ross* + USCO-2025, is **US fair use for the extraction step** defensible if the dataset could compete in a "data/analytics" market ? does publishing only numbers (never footage) matter?
173   3. India: are anti-scraping ToS enforceable; **IT Act s.43/s.66** exposure for any 3rd-party ingestion; does pending *ANI v. OpenAI* (~Apr 2026) move it? Confirm **DPDP s.9/Rule 10** and whether per-player tracking is prohibited s.9(3) monitoring of a minor.
174   4. Sign-off on the **final committed Tier-0 subset + provenance log**; is non-attributability-via-private-source a defensible posture to assert given its conditional/revocable nature?
175
176   **Owner (strategic):**

```
177     5. Publishing the Fork-A subset **forfeits the trade-secret moat** (Indian breach-of-
confidence framing) ? weighed against community/benchmark/paper aims. Your call.
178     6. **Forward-looking commitment:** do not publish a highlight reel of the *same* source
whose fixtures are committed (without re-clearing), and accept a periodic re-assessment as the
public-video environment grows.
179     7. **Scope of first public commit:** synthetic-only (zero provenance risk, recommended)
vs a counsel-cleared real subset.
180     8. **Tier-1 mechanism:** out-of-repo+token (recommended) vs SOPS+age in-repo. And: is
protobuf (O1) wanted, or is canonical-JSON enough?
181
182     ---
183
184     ## 9. Definition of done
185
186     The project is **DONE** (flip `Status ? DONE`, archive) when:
187     - [ ] **P0, M1?M4, P1, DOC** are ? and merged.
188     - [ ] A clean `git clone` on **win + ubuntu + mac** runs `python run_all_tests.py` and
the **Tier-0 gates pass green with no key / no video / no GPU / no DB**.
189     - [ ] The **CI leak-scan is a required status check** and passes (no MD5 / real-name /
non-surrogate / plaintext-SOPS in the tree).
190     - [ ] Characterization + quality-floor gates are **parse-compared, CONTROL-only,
windowing-fingerprinted**, and demonstrably catch a seeded behaviour change.
191     - [ ] `docs/CODE_MAP.md §0` + TEST MAP updated; this doc cross-linked; `NEXT_STEPS.md` +
memory `current-state` point here.
192     - [ ] **Either** the public commit is synthetic-only, **or** (**P2 owner-accepted ? +
P3**) for a real-data commit, under the §3 conditions enforced **in code** by P1's refusal gate.
193     - [ ] §6 honest-limits caveat rides in the test pass/fail messages, not only the docs.
194
195     *Optional O1/O2 may remain ? without blocking DONE (they're earned, not assumed).*
196
197     ---
198
199     ## 10. References
200
201     **Internal code anchors (verified):** `backend/eval/{regression.py,fusion_golden.py,fusi
on_compare.py,ablation.py,experiment.py,metrics.py,windowing.py,distill_local.py,gt_loader.py,ev
al_stats.py}` · `backend/pipeline/detectors/{base.py,tracknet_runner.py,rally_gate.py}` ·
`backend/utils/hashing.py::compute_video_id` · `eval_baselines/{heuristic_lovo_n6.json,gemini_wr
apper_2026-06-10.json,experiment_leaderboard.json}` ·
`tests/test_ablation.py::test_litmus_reproduces_gate_a` · `.github/workflows/ci.yml`.
202     **Internal docs:** `GOLDEN_DATA_SHARING.md` (#175), `EVALUATION.md`,
`EXPERIMENT_HARNESS.md`, `QUALITY_ITERATIONS.md`, `MULTI_SIGNAL_FUSION_PLAN.md`, `CODE_MAP.md
§0`.
203     **Legal authorities:** see §3 (full URLs in the research workflow artifact). *This
section is researched general information, not legal advice.*
204
205     ---
206
207     ### Changelog
208     - **2026-06-16** ? Initial draft synthesized from two multi-agent workflows (mechanics
design + 2 critiques; legal memo IN/US/EU + SEALED-FIXTURES architecture + red-team). Verified
the live name-leak and the BLAS-determinism cause against code. Status: DRAFT, owner-gated.
209
```

28. assistant (2026-06-16T06:27:57.299Z)

Now I have a comprehensive view. Let me verify a few critical determinism/correctness suspicions empirically. First, the key over-claim: the doc §4a (line 80) claims the replay uses `windowing.build_windows(preset)` and `rally_gate net_crossings`, with `min_crossings`. Let me verify the `net_crossings` round-trip stability and the `_fmt`/`parse_timestamp` round-trip for the real-fixture reset. Let me also check the README and the `.gitattributes` LF pin claim.

29. user (2026-06-16T06:28:01.122Z)

=== README ===

`eval_baselines/fixtures/` ? Golden Regression Fixtures (Tier-0, synthetic)

Video-free, **CONTROL-only** characterization fixtures for the rally-segmentation pipeline. A contributor on **any machine ? no video, GPU, WSL, or DB** ? replays these committed structured files through the already-shipped eval stack and gets a red/green regression signal. See `docs/GOLDEN_REGRESSION_FIXTURES.md` (deliverable **M1**; §4, §6, §7).

What's here

Each fixture is one directory:

File	Purpose
`trajectory.csv`	Synthetic shuttle trajectory (`Frame,Visibility,X,Y`) ? the freeze seam input.
`labels.csv`	Synthetic ground-truth rally intervals (`start,end`).
`expected.json`	Frozen replay snapshot (candidates + 5 shuttle features + CONTROL segments + score).
`provenance.json`	Pinned windowing params + privacy/licensing tags + description.

`manifest.json` lists every fixture; `backend/eval/golden_fixtures.py` is the shared replay module + freeze/refresh/check CLI.

Guardrails (non-negotiable)

- **Synthetic / owned only.** Every fixture is generated by deterministic formulae (NO random) ? zero third-party footage, zero provenance/licensing risk. Tagged `kind: "synthetic"`, `license: "synthetic"`, `minors_free: true`.
- **No biometric / identity streams.** Tier-0 carries the **5 shuttle feature columns only** (`duration, peak_velocity, mean_velocity, active_ratio, net_crossings`). There is, by design, **no person / pose / gait / face / audio / player field anywhere in the schema** ? a test asserts this positively (deletion-at-source, §4c). Minors and biometrics are excluded.
- **CONTROL / pure-stdlib only.** The replay uses `experiment.CONTROL` (`is_learned() == False`) ? no numpy / logistic-regression. Learned variants' near-threshold decisions flip across CPUs, so they can **never** anchor a cross-machine snapshot (§4d / D5).
- **Parse-compare, never byte-compare.** The gate `json.load`s both sides: exact equality on int/structural fields, abs-tolerance `1e-6` on floats (CRLF/float-format safe).
- **Windowing is pinned per fixture.** The `(vt, mg, mwd, max_win)` params live in each `provenance.json` and are passed explicitly to `build_candidates` ? the replay does **not** depend on the named `windowing.SERVED` constant, so a future SERVED retune can't silently churn fixtures.

Fixtures are GENERATED, never hand-edited

Do **not** hand-edit `expected.json` / `provenance.json` / the CSVs. They are produced correct-by-construction by the CLI.

```bash
# Generate the synthetic trajectory.csv + labels.csv for all built-in scenarios, then freeze expected.json + provenance.json + (re)write manifest.json:
python -m backend.eval.golden_fixtures --freeze-synthetic

# Replay all fixtures and parse-compare vs the committed expected.json (exit 0/1):
python -m backend.eval.golden_fixtures --check

# Re-freeze expected.json from the COMMITTED trajectory+labels (NO regeneration). ? T
# rubber-stamp a real regression ? review the visible diff and record a reason (§6):
python -m backend.eval.golden_fixtures --refresh-snapshot --all  --reason "<why>"
python -m backend.eval.golden_fixtures --refresh-snapshot --slug clean_single_rally --reason "<why>"
```

```

### Honest limits (§6 ? rides in the test pass/fail messages, not only here)

- A green suite proves only that the **deterministic post-trajectory transforms** are unchanged
  for the frozen cases. It does **not** cover the WASB/TrackNet detector, frame decoding, the
  AI/provider handoff, chunking, the ffmpeg stitch, DB persistence, or the API filter.
- Characterization is **behaviour-locking, not correctness** ? it freezes current behaviour
  *including current bugs*. "Passing" = "behaviour unchanged", never "behaviour correct".
- **Synthetic-tier F1 measures plumbing correctness, NOT field quality.** Never cite a synthetic
  fixture's F1 in `QUALITY_ITERATIONS.md` or the paper as rally-detection performance.
=== .gitattributes (fixtures-related) ===
# Golden Regression Fixtures (M1) ? pin LF line endings so the committed fixtures are (never byte-compares) JSON, but a stable on-disk form keeps `git diff` reviewable and fixtures reproducible. See docs/GOLDEN_REGRESSION_FIXTURES.md §4d/§4e.
eval_baselines/fixtures/** text eol=lf
=== git check-attr on a fixture ===
eval_baselines/fixtures/clean_single_rally/trajectory.csv: text: set
eval_baselines/fixtures/clean_single_rally/trajectory.csv: eol: lf
eval_baselines/fixtures/clean_single_rally/expected.json: text: set
eval_baselines/fixtures/clean_single_rally/expected.json: eol: lf

**30. assistant (2026-06-16T06:28:04.720Z)**

Good, the .gitattributes LF pin is real and applies. Now let me empirically verify the most important determinism suspicions. I'll check whether numpy is actually installed here (to know if I can run the replay), and verify the `net_crossings` round-trip and the `_round`/0.0 normalization edge.

**31. user (2026-06-16T06:28:10.900Z)**

numpy 2.4.6
---import golden_fixtures w/o numpy test (simulate clean clone)---
IMPORT FAILED without numpy: ModuleNotFoundError No module named 'numpy'

**32. assistant (2026-06-16T06:28:15.715Z)**

Confirmed empirically: `import golden_fixtures` fails without numpy because of the transitive `experiment ? classifier ? numpy` chain. This is a real over-claim against the docstring ("pulls in no numpy/cv2 on the CONTROL path", line 35) and the README ("CONTROL / pure-stdlib only ... no numpy"). The CI installs numpy so it's green, but the "no numpy" framing for *import* is false ? only *execution* avoids numpy (which is the already-verified caveat).

Now let me verify the determinism cliff suspicions empirically: net_crossings round-trip, and the time-origin reset float round-trip for real fixtures.

**33. user (2026-06-16T06:28:21.862Z)**

clean_single_rally net_crossings= 7 int
multi_rally_with_deadtime net_crossings= 7 int
multi_rally_with_deadtime net_crossings= 7 int
occlusion_break net_crossings= 7 int
occlusion_break net_crossings= 7 int

**34. assistant (2026-06-16T06:28:27.202Z)**

net_crossings is computed via `count_net_crossings` which counts integer sign changes ? it's a pure integer count from the start (`g.crossings` is `int` in `GatedWindow`), and `int(round(...))` in golden_fixtures is just defensive. Stable. Now the key determinism concern from the lens: net_crossings depends on `_median`/`_spread` and `deadband` over the trajectory, and the trajectory frames go through `frame*fps` float arithmetic in `_points_in`. Let me trace
```

the actual `frame*fps` round-trip concern and check the time-origin reset round-trip for real fixtures (the de-attribution path the CI does NOT test).

35. user (2026-06-16T06:28:39.540Z)

```
labels written:
start,end
0.233333,4.233333

score equal: False
on score: {'iou_threshold': 0.5, 'num_pred': 1, 'num_gt': 1, 'true_positives': 1,
'false_positives': 0, 'false_negatives': 0, 'precision': 1.0, 'recall': 1.0, 'f1': 1.0,
'mean_iou': 0.983333, 'mean_start_error': 0.033333, 'mean_end_error': 0.033333}
off score: {'iou_threshold': 0.5, 'num_pred': 1, 'num_gt': 1, 'true_positives': 1,
'false_positives': 0, 'false_negatives': 0, 'precision': 1.0, 'recall': 1.0, 'f1': 1.0,
'mean_iou': 0.983333, 'mean_start_error': 0.033334, 'mean_end_error': 0.033333}
on segs: [[0.033333, 3.966667]]
off segs: [[0.266667, 4.2]]
reset labels.csv:
start,end
0.000000,4.000000
```

36. assistant (2026-06-16T06:28:45.880Z)

This is a real finding. The doc and code claim the time-origin reset is "**provably metric-invariant** ? bit-identical scores" (doc §4c line 94, code comment line 620-623, the M2 NOTE). But empirically, with a non-integer-friendly offset, `mean_start_error` differs: `0.033333` (with reset) vs `0.033334` (without reset). This is a floating-point round-trip artifact: the labels.csv stores 6dp-rounded seconds, and the trajectory is rounded too, so the subtraction is NOT bit-exact when t0_sec is a non-terminating decimal.

The reason the committed test passes (`test_time_origin_reset_is_metric_invariant`) is that it uses `offset_frames = 9000` ? `9000/30 = 300.0` exactly, an integer number of seconds, so no rounding cliff is hit. Let me confirm the mechanism precisely and check whether this can flip a 6dp digit / cross the 1e-6 boundary.

37. user (2026-06-16T06:28:58.863Z)

```
offsets tested=40 metric-invariance-BROKEN=26  worst |delta|=1.000000000001e-06
NOTE: claim is bit-identical scores; any broken count contradicts the over-claim.
```

38. assistant (2026-06-16T06:29:04.603Z)

This is significant: 26 of 40 sub-second offsets break the "bit-identical score" invariance claim, and the worst delta is `1.000000000001e-06` ? which is just over the `_FLOAT_TOL = 1e-6` boundary used by `compare_expected`. This is exactly the "rounding cliff that flips a 6dp digit cross-OS" suspicion, and it sits right on the 1e-6 comparison boundary.

Let me confirm whether this can actually cause a `compare_expected` failure (i.e., whether the worst delta crossing 1e-6 would make `--check` go red after a re-freeze on a different machine), and pin down the exact mechanism ? it's the asymmetry between `_reset_labels` (subtract first, then `_fmt`-round on write) vs the non-reset path (round on write only).

39. user (2026-06-16T06:29:10.469Z)

```
compare_expected approx uses abs tol 1e-6; |delta| here = 1.000000000001e-06
is that > 1e-6? True
worst-measured 1.000000000001e-06 > 1e-6 -> True
```

40. assistant (2026-06-16T06:29:18.648Z)

The mechanism is confirmed. This is a real determinism/correctness gap in the de-attribution path, masked by the test's choice of a whole-second offset. Two distinct issues:

1. The "metric-invariant / bit-identical" claim is **false** for non-whole-second `t0` offsets (the common real-world case, since real footage rarely starts on a whole second).
2. The worst delta lands exactly on the `1e-6` comparison cliff ? so it CAN flip a `compare_expected` digit/result.

Crucially, this de-attribution path (`freeze_real_fixture`) and its test file `tests/test_golden_real_fixtures.py` are **NOT** in the golden-crossos CI job ? confirming the lens's key suspicion that the de-attribution/reset path is cross-OS-unguarded.

Let me now verify the remaining lens items quickly: dict/set iteration in compare_expected/JSON writers (sort_keys), and whether the `_scan_forbidden` source-path scan has a real hole. Let me also double check the `gt_hash` rounding (3dp) interplay with the reset.

41. user (2026-06-16T06:29:27.127Z)

```
offset=7 gt_hash on=6c93801fd607 off=7b35e330c021 equal=False
offset=11 gt_hash on=6c93801fd607 off=81b6150f1925 equal=False
offset=13 gt_hash on=6c93801fd607 off=1326768c9e6a equal=False
```

42. assistant (2026-06-16T06:29:36.161Z)

That's expected and correct (gt_hash *should* differ because the reset deliberately shifts the timeline ? gt_hash is a fingerprint of the absolute label values, and the reset's whole point is to sever the absolute timeline). Not a bug ? the reset gt_hash is internally consistent (the on-version always hashes the reset labels). Good.

Now let me verify the iteration-order determinism in the JSON writers and compare_expected (sort_keys), and check one more potential real edge: negative t0 / empty points in the reset path, and whether `freeze_real_fixture` validates that all frames are non-negative after reset.

43. user (2026-06-16T06:29:47.781Z)

```
sort_keys in _write_json: True
CRASH: ValueError C:\Users\avidu\AppData\Local\Temp\tmp5rnbe8px\out\fx_neg\labels.csv:2:
negative start (-3.333333)
```

44. assistant (2026-06-16T06:29:56.603Z)

Another real edge-case finding: if a GT label starts before the first trajectory frame, the time-origin reset produces a negative label start, and the STRICT re-parse in `replay_fixture` crashes with a `ValueError` (`negative start`) ? but only AFTER `freeze_real_fixture` has already written `trajectory.csv`, `labels.csv`, and `provenance.json` to disk (the crash is at line 673 `replay_fixture(fdir)`, after writes at lines 635-636 and 671). So a partial/orphaned fixture directory is left on disk on this failure. This is a lower-severity robustness gap (mismatched trajectory/label time bases would be a labeling error), but it's a real un-tested path: the crash message also surfaces the temp source path, though it's a runtime exception not a committed artifact.

Let me verify the JSON writer determinism is complete and check one final item ? whether the `--check` CI path actually re-derives net_crossings deterministically across the float frame arithmetic in `_points_in` (the `start*fps <= p.frame` comparison). Let me also confirm the committed `active_ratio > 1.0` oddity is benign and look at whether `compare_expected` misses any field (over-claim on coverage).

45. user (2026-06-16T06:30:04.491Z)

```
top-level written: ['candidates', 'control_segments', 'gt_hash', 'iou', 'schema_version',
'score', 'slug']
top-level NOT compared: []
score keys NOT compared: []
candidate keys: ['end', 'shuttle', 'start'] (compare_expected checks start,end,shuttle)
```

46. assistant (2026-06-16T06:30:08.254Z)

`compare_expected` covers every written field ? that part is sound, no coverage hole there. Now let me confirm the key CI-coverage gap precisely: that `test_golden_real_fixtures.py` is genuinely absent from every CI job (not just golden-crossos), and check whether the full-suite job (ubuntu-only) runs it.

47. user (2026-06-16T06:30:10.261Z)

```
90:      # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
97:        run: python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-
fail-under=70
122:          python -m pytest tests/test_golden_fixtures.py -q
```

48. user (2026-06-16T06:30:13.442Z)

```
run_all_tests mentions tests dir / pytest discovery:
   A single, discoverable entry point so you (or a reviewer / CI) can verify the
   python run_all_tests.py             # full suite, quiet
   python run_all_tests.py -v          # verbose (passthrough to pytest)
   python run_all_tests.py -k config   # filter by keyword (passthrough)
   python run_all_tests.py --cov=backend --cov-report=term-missing   # coverage
   Anything after the script name is passed straight through to pytest.
   - The observability **reporting** tests skip cleanly if the optional ``obsreport``
   package isn't installed (they use ``pytest.importorskip``). To exercise them too:
   - Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
   cmd = [sys.executable, "-m", "pytest", *args]
   obs = "available" if _installed("obsreport") else "NOT installed ? reporting tests will SKIP"
   print(" RESULT: " + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit {rc}"))
```

49. assistant (2026-06-16T06:30:22.109Z)

Confirmed. The picture is clear:

- `test_golden_real_fixtures.py` (the de-attribution / `freeze_real_fixture` tests) IS run by the **ubuntu-only** `full-suite` job (via `run_all_tests.py` ? pytest discovery on `tests/`), but is NOT in the `golden-crossos` matrix (windows/macos). So the de-attribution/reset path has **zero cross-OS coverage** ? it's only ever exercised on ubuntu.
- The `golden-crossos` job runs only `--check` + `test_golden_fixtures.py` (synthetic, whole-second `t0=0`), which never exercises the non-whole-second-`t0` reset path that breaks the metric-invariance claim.

This means the metric-invariance bug I found would only surface cross-OS if/when a real fixture with a fractional-second start is committed (P3, currently gated), and even then only on the ubuntu lane ? but the doc's "bit-identical / provably metric-invariant" claim is an over-claim regardless.

Let me do one final confirmation on the precise cross-OS angle of the lens: whether the synthetic CONTROL path itself (the committed fixtures) has any genuine cross-OS float risk, or whether the `frame*fps` arithmetic is integer-exact for these fixtures. This determines whether the *committed* path is truly safe (as already-verified) vs whether I'm contradicting the already-verified item.

50. user (2026-06-16T06:30:26.318Z)

```
frame=1 sec=0.03333333333333333 sec*fps=1.0 exact=True <=frame? True
frame=7 sec=0.23333333333333334 sec*fps=7.0 exact=True <=frame? True
frame=119 sec=3.966666666666667 sec*fps=119.0 exact=True <=frame? True
frame=209 sec=6.966666666666667 sec*fps=209.0 exact=True <=frame? True
frame=210 sec=7.0 sec*fps=210.0 exact=True <=frame? True
```

51. assistant (2026-06-16T06:30:33.782Z)

For these committed fixtures, `(frame/fps)*fps` round-trips exactly (IEEE-754 double rounds back to the integer), so net_crossings is stable ? the committed CONTROL path is genuinely cross-OS-safe, consistent with the already-verified item. But note this relies on the `start`/`end`

passed to `_points_in` being the *unrounded* window bounds. In `replay_fixture`, `build_candidates` returns `intervals` (unrounded), and those unrounded intervals feed `predict`/`score` ? the rounding to 6dp only happens at JSON-write time. So the computation path is unrounded; good. The boundary `start*fps <= p.frame` is on the IEEE-exact value. This corner is clean for the committed fixtures.

I now have enough to write the audit. Let me do a final scan for whether there's a `core.autocrlf` interaction risk with the `.gitattributes eol=lf` pin that could matter for the trajectory.csv re-read on Windows (the parse_trajectory_csv opens with default newline handling).

52. user (2026-06-16T06:30:38.579Z)

parse_trajectory_csv newline arg: 1
OK: csv.reader + json.load are line-ending agnostic; eol=lf pin + parse-compare cover CRLF.

53. assistant (2026-06-16T06:30:43.942Z)

I have completed a thorough audit. Let me compile the findings into the structured output.

Summary of what I verified:
- **CONTROL committed path (synthetic fixtures):** genuinely cross-OS-safe ? `(frame/fps)*fps` round-trips exactly, net_crossings is a pure int count, JSON writers use `sort_keys=True`, `compare_expected` is order-independent and covers every written field, CRLF is handled by `csv.reader(newline='')` + `json.load` + the `eol=lf` pin. This corner is clean (consistent with the already-verified items).
- **Real findings** (what per-PR verification missed) below.

54. user (2026-06-16T06:31:36.895Z)

Structured output provided successfully

55. assistant (2026-06-16T06:31:47.114Z)

I've completed the determinism / cross-OS audit of the Golden Regression Fixtures feature and returned my findings via StructuredOutput.

Key results:

The committed synthetic CONTROL path is genuinely safe ? I honestly confirmed the lens's specific suspicions about it are clean: `(frame/fps)*fps` round-trips integer-exact for these fixtures so `net_crossings` is stable, JSON writers use `sort_keys=True`, `compare_expected` is order-independent and covers every written field, and CRLF is fully handled by `csv.reader(newline='')` + `json.load` + the `eol=lf` `.gitattributes` pin.

But the lens's central suspicion was correct, and I found a real defect cluster on the de-attribution path that per-PR verification missed:

1. **HIGH** ? The time-origin reset's documented "provably metric-invariant / bit-identical scores" guarantee is false for any non-whole-second `t0`. I broke it on 26/40 sub-second offsets; worst score delta `1.0000000000001e-06`, landing exactly past the `1e-6` `compare_expected` tolerance cliff. The committed invariance test only passes because it hard-codes a whole-second (9000-frame / 300.0s) offset.

2. **MEDIUM** ? `tests/test_golden_real_fixtures.py` (the entire de-attribution/reset/M2 path) is NOT in the `golden-crossos` matrix ? it runs ubuntu-only via the full-suite lane. So that float-sensitive path has zero cross-OS coverage, exactly as the lens suspected.

3. **LOW** ? `freeze_real_fixture` crashes after partial on-disk writes when a GT label precedes the first trajectory frame (negative reset start rejected by the strict re-parse), leaving an orphaned fixture dir.

4. **LOW** ? The "pure-stdlib / no numpy on the CONTROL path" framing over-claims for *import*: `import golden_fixtures` fails on a clean clone without numpy (transitive `experiment ?

classifier` numpy import). Only numpy *execution* is avoided; CI stays green only because it installs numpy.

Verdict: NOT CLEAN. None of these break the currently-committed synthetic fixtures, but they undermine the safety claims for the real-fixture (P3) path and the §9 "win+ubuntu+mac" definition-of-done.